List of Potential Projects Data and Text Mining Course 2026

LINK FOR PROJECT ALLOCATION -→
https://1drv.ms/w/s!AtcJs3OTsMZuiSApqWJaa2SDtz2Y

Project 1: Keywords Selection and Analysis 1

This project aims to test various methodologies for identifying appropriate keywords from the text as appropriate representation of the content of the document.

We consider three types of algorithms for selecting from document: TF-IDF, RAKE (Rapid Automatic Keyword Extraction based on word co-occurrence), YAKE (Yet Another Keyword Extractor relying on statistical characteristic of the text), TextRank (that uses graph representation and pagerank).

Student may refer to existing implementations in NLTK, sciki-learn for TF-IDF, implementation available in https://github.com/LidaAleksanyan/spatial_keyword_extraction for RAKE, YAKE. Luhn and other existing medium.com implementation of popular keyword extraction algorithms in natural language processing.

1. Write a script that allows you to collect 1000 documents, which include a summary of paper, its title, and list of keywords as entered by the authors, from PubMed / Medline with a focus to biomedical papers, and save the results as a csv database D containing the three attributes (title, abstract, keyword lists).
2. Write a script that extracts the 10 most dominant keywords (if available) for each summary document using TF-IDF, RAKE, YAKE and TextRank algorithms, and save the results in database D.
3. Write a script that calculates the precision, recall and F1 scores taking into account the reference keyword list inputted by the authors and available in database D. Summarize the result in a table comparing the performance of each of the algorithms TF-IDF, RAKE, YAKE, TextRank. Then plot the corresponding AUC. Note: the calculus of precision, recall and F1 should take into account the number of keywords available in D and how to accommodate the 10 generated keywords for this purpose. Students should suggest their own reasoning to accommodate this constraint.
4. We want to utilize the title of the document as an additional proxy for the selection of the keywords. For this purpose, write a script that extracts 20 most dominant keywords (if available) using the four algorithms, and then filter these 20 generated features using string matching only (through the use of FuzzyWuzzy method), to yield 10 keywords (if available). Calculate the new Precision, Recall and summarize the finding in a table, and plot AUC.

5. Repeat 4) when using the semantic similarity between each keyword candidate; namely, for each keyword candidates one calculates the semantic similarity with each token of the title and take the maximum over all tokens of the title. Summarize the new results in terms of Precision and Recall in a new table, and plot the corresponding AUC.
6. Repeat 5) where instead of using semantic similarity, we use word2vec embedding, so the cosine similarity between the word2vec embedding of each keyword candidate and each token of the sentence (maximum over all tokens of time is taken as the similarity of the candidate keyword and title).
7. Suggest a machine learning-based approach (supervised) that would allow you to use the keywords inputted by authors as training labels and devise a machine learning pipeline for predicting the keywords on the testing sample (20% of the overall database D).
8. Now we want to implement updated state-of-the-art algorithms. For this purpose, we want to use the SEKE (Specialized Experts for Keyword Extraction) Keyword extraction that uses DeBERTa backbone coupled with a mixture-of-experts (MoT) routing layer – see an example of its implementation https://github.com/matejMartinc/SEKE_keyword_extraction. Test SEKE algorithm on the database D, and summarize the result in a table and AUC plot.
9. Feel free to use your innovation and own search on state-of-the-art keywords extraction models to perform the same task.
10. Test the developed algorithms on more elaborated benchmark datasets, e.g., GitHub - INESCTEC/KeywordExtractor-Datasets · GitHub. Select three dataset from the list available in the above link and summarize the results of the various algorithms in terms of Precision, Recall, F1 measure, and AUC plot.
11. Identify relevant literature in keyword extraction from open dataset and provide a concise and critical summary of the relevant literature and state of the art in the report.

Project 2: Keywords Selection and Analysis -Topical Analysis 2

This project aims to test various methodologies for identifying appropriate keywords from the text as appropriate representation of the content of the document, while taking into account the topics of these documents.

We consider three types of algorithms for selecting from document: TF-IDF, RAKE (Rapid Automatic Keyword Extraction based on word co-occurrence), YAKE (Yet Another Keyword Extractor relying on statistical characteristic of the text), TextRank (that uses graph representation and pagerank), TopicRank and its variants (e.g., Sterckx et al., 2015 "Topical word importance for fast keyphrase extraction", Wan and Xiao (2008), "Collabrank: towards a collaborative approach to single-document keyphrase", PositionRank.

Student may refer to existing implementations in NLTK, sciki-learn for TF-IDF, implementation available in https://github.com/LidaAleksanyan/spatial_keyword_extraction for RAKE, YAKE. Luhn and other existing medium.com implementation of popular keyword extraction algorithms in natural language processing. See also implementation in https://github.com/boudinfl/pke/tree/master/pke/unsupervised/graph_based for TopicRank and its variants as well as positionRank algorithms.

1. Write a script that allows you to collect 1000 documents, which include a summary of paper, its title, and list of keywords as entered by the authors, from PubMed / Medline with a focus to biomedical papers, and save the results as a csv database D containing the three attributes (title, abstract, keyword lists). The extraction should be such that 10 distinct topics will be selected originally, and then collect 100 abstracts (together with their titles and authors' keyword list) per each topic. Note: use your own grounded methodology to select documents belonging to same topic through query-focused approach.
2. Write a script that extracts the 10 most dominant keywords (if available) for each summary document using RAKE, YAKE, TopicRank and its variants in the above the above link (singleRank, ExpandRank, TPR, TextRank) and PositionRank, and save the results in the constructed database D.
3. Write a script that calculates the precision, recall and F1 scores taking into account the reference keyword list inputted by the authors and available in database D. Summarize the result in a table comparing the performance of different algorithms for each topic. Note: the calculus of precision, recall and F1 should take into account the number of keywords available in D and how to accommodate the 10 generated keywords for this purpose. Students should suggest their own reasoning to accommodate this constraint.
4. Consider the mean reciprocal rank (MRR) curves defined as the averaged ranking of the first corrected prediction: $MRR = \frac{1}{|T|}\sum_{d\in T}\frac{1}{r_d}$

   Where T is the list of all documents and r_d is the rank at which the first correct keyphrase of document d was found.

Draw the MRR curves for TopicRank, PositionRank, SingleRank, ExpandRank, TPR, TextRank, RAKE, YAKE.

5. We want to study the performances of the above algorithms at topic level. For this purpose, we want initially to test the hypothesis that the selected 100 abstract belong to the same topic. In this respect, construct the BERT embedding of each document (abstract text) and then calculate the inner product of the embedding vectors of all 100 documents; this yields the consistency v for each topic. Then draw a plot showing the evaluation of each performance (precision, recall, F1) with respect to topical consistency v.
6. Repeat 5) when the topical consistency is constructed using the cosine similarity of the BERT embedding vectors of the titles of the documents on the same topic.
7. Repeat 5) when the topical consistency is constructed using Latent Dirichlet model. For this purpose, assume for instance LDA with one topic and 10 keywords per topic. Then evaluate the topical consistency as the percentage of common keywords of all LDA generated keywords (for documents belonging to same topic) over the total number of distinct keywords of all LDA generated keywords.
8. Discuss whether the topical consistency can be used as a proxy to enhance the relevancy of keyword identification using the methods in 2)
9. We want to utilize the title of the papers to enhance the performance of the keyword extraction. From the methods suggested in Project 1, test a set of candidates solutions and report the findings. Calculate the corresponding Precision, Recall, F1 scores and summarize the results in the corresponding table (s).
10. Suggest a machine learning approach (supervised approach) where the inputted keywords, topic label (to distinguish documents of the same topic) are used to learn the keywords. Use 80% of dataset to train the model and 20% to test the model.
11. Use your own literature search to identify state-of-the-art transformers approaches to integrate topical information and keyword information extraction and test the model on the database D. Compare the results with models in 2).
12. Identify relevant literature in keyword extraction from open dataset and provide a concise and critical summary of the relevant literature and state of the art in the report.

Project 3: Keywords Selection and Analysis -LLM Analysis 3

This project aims to test various methodologies for identifying appropriate keywords from the text as appropriate representation of the content of the document, while taking into account the topics of these documents. We consider in this project more state-of-the-art techniques issued from transformers, notably KeyBART model (https://huggingface.co/bloomberg/KeyBART) and LLMs-based reasoning tested on publicly available databases; namely, Inspec, kp20k, nus, accessible in huggingface:

https://huggingface.co/datasets/midas/inspec

https://huggingface.co/datasets/midas/kp20k

https://huggingface.co/datasets/midas/nus

1. Consider an existing implementation of KeyBART from e.g., Hugginface or elsewhere, and consider a paper abstract on your own (extracted from academic paper or from news document). Write a script that displays the abstract document and highlights in yellow the keyphrases generated by KeyBART (up to 10 keyphrases). Include a screenshot of the illustration in the project report, and save the result in a database D.
2. We want to test the extent to which the generated keyphrases are diverse and independent from each others. For this purpose, compute the Self-BLEU score, by computing the BLEU score of each phrase against all other phrases as references (lower Self-BLEU score higher diversity).
3. Repeat 2) while the diversity is measured using embedding dispersion. For this purpose, generate the Sentence-BERT embedding vector of each KeyBART generated phrase. Then calculate the cosine distance between each pairs of embedding vectors (higher cosine distance indicates higher diversity). Next report the minimum, maximum and average cosine distance among all pairs of KeyBART embedding vectors. Finally, use the t-SNE (t-distributed Stochastic Neighbor Embedding) to visualize each embedding vector as a point in 2D space.
4. Repeat 3) while instead of using cosine distance of the Sentence-BERT embedding, we use HDBSCAN of embedding vectors to cluster the corresponding embedding vectors. The number of clusters generated by HDBSCAN algorithm is an indicative measure of the structural diversity (higher the number of clusters, higher the structural diversity).
5. We want to repeat the operations 2-4) while using alternatives to KeyBART such as CatSeqm CatSeqD, CatSeqCorr, ExHiRD-h models as implemented in GitHub - Chen-Wang-CUHK/ExHiRD-DKG: The source code for ACL 2020 paper Exclusive Hierarchical Decoding for Deep Keyphrase Generation · GitHub
6. Consider the Inspect, kp20K and nus databases highlighted above. Use the implementations of KeyBART, CatSeqm, CatSeqD, CatSeqCorr, ExHiRD-h for generating the appropriate keyphrases and then use the ground truth knowledge in the databases to estimate, Precision, Recall and F1-score.

7. We want to enhance the KeyBART by utilizing the title information, whenever available. For this purpose, study the implementation https://doi.org/10.1609/aaai.v33i01.33016268 for integrating the title information in the keyphrase generation. Suggest and implement a simple heuristic that extends KeyBART to take into account the title information. Then evaluate the performance of this algorithm on some of the databases mentioned above in terms of Precision, recall and F1 measure.
8. We want to use a simple LLM model of your choice using simple vanilla prompt to generate appropriate Keyphrase. Evaluate the performances of the model on Inspec and kp20k database in terms of Precision, Recall and F1 measure.
9. Use your own research and investigation to enhance the LLM model by integrating, e.g., advanced Prompting, LORA fine-tuning, zero versus few shot learning, etc.. and test the evaluation of the new model on the three databases.
10. Identify relevant literature in keyphrase extraction from LLMs literature and provide a concise and critical summary of the relevant literature and state of the art in the report.

Project 4: Keywords Selection and Analysis -Poems Analysis 4

This project aims to test the keyword extraction in poetry text taking into account the specialized linguistic components of such as prosody (rhythm, stress, intonation of speech) instead of treating words as isolated tokens. We consider the prosody implementation available at https://github.com/Helsinki-NLP/prosofy. On the other hand, we consider available poems in Gutenberg collection of open books, available in NLTK. Select a poem document of your own choice from Gutenberg database.

1. Follow the recommended guidelines in help file to install prosofy and write a script that highlights prosodic prominence, stress and intonation from a small sample of poem from Gutenberg (feel free to select an example on your own). Ideally, a small interface will shows the outcome will be displayed.
2. Poems are sometimes organized in chapters, or subsections or subsets. The distinction between the various chapters/subsections differ from one poem to another one. After manual scrutinization of the selected poem, suggest a NLP construction that would allow you to distinguish between the different chapters automatically.
3. We want to determine what linguistic patterns are present in the poem or each chapter of the poem. For this purpose, write a script that tests what a single line of poem / chapter has in common? More specifically, it tests i) the presence of identical words, ii) identical normalized word (token after lemmatization stage), iii) total number of words, iv) number of characters, v) number of PoS tags, iv) presence and number of polarity words, vi) entity relationship (according to WordNet identified relationship), vii) types of verbs employed (action verbs, stative verbs, auxiliary verbs, modal verbs), similarly type of pronouns, types of adverbs, type of adjectives; vii) presence of synonymy, antonymy (through use of external dictionary of synonymy, antonyms). Show which linguistic patterns hold for each of (chapter of) poem, and how these patterns evolve from one chapter to another.
4. Test and comment whether prosofy may help in identifying some of the above linguistic patterns.
5. We want to evaluate the keywords extraction-based approach. For this purpose, consider groups of five successive of lines as a single cluster (other more elaborated clustering can also be suggested). We then consider the three most dominant keywords according to YAKE, TextRank and TopicRank. Save the results for each cluster in a database.
6. Now we want to visualize the results of 5) across all clusters belonging to the same chapter of the poem. For this purpose, generate the word2vec embedding of each generated keyword by the three algorithms, then use the PCA to project the embedding space into two dimensional space, so that each keyword is represented as a point in this cartesian space. Study the evolution of the three points generated by each algorithms across all clusters of the chapter (especially check whether some curve fitting can be established to explain the evolution of the keywords). Suggest your own performance metrics to compare the performance of the three algorithms.

7. Use SentiWordnet to calculate the sentiment polarity of each line of poem. Then use a triangle representation (where nodes stand for positive, negative and neutral) to represent each line polarity as point in this triangle. Then display the sentiment points for all lines of the same chapter. Discuss how chapters convey the sentiment polarity in overall.
8. Instead of SentiWordNet, use the NRC-VAD lexicon (Saif | VAD Lexicon) and NLTK to calculate the average valence and average arousal for each line of poem. Now consider a 2D space where the x-coordinate is the average arousal score and y-coordinate is the average valence score of a given poem line. Display the evolution of the various lines in terms of valence-arousal for each chapter, and study weather some trend can be inferred from the plot
9. Explore HuggingFace transformers library and identify an implementation that generates valence and arousal scores for each poem line of each chapter. Display similar plot as in 8) and study possible trend.
10. Suggest any additional contribution / implementation that you may judge doable to show some observable trends in poetry analysis using advanced transformer structures.
11. Identify relevant literature in computational poetry analysis and provide a critical concise summary of the most relevant works.

Project 5: Stopword behavior in Poetry analysis 1

This project aims to explore the occurrence of stopword list in poetry. We first consider the default English stopword list in NLTK, called, say, *L*. Also, consider available poems in Gutenberg collection of open books, available in NLTK. Select three poem documents of your own choice from Gutenberg database.

1. Considering the default NLTK stopword list L, write a script that identifies stopwords present in the selected poem and draws frequency plot. Study whether a power of law distribution can be fit by estimating the goodness of fit. Repeat this process for each poem document.
2. We want to study how much the stopword list L is representative in each of the poems. For this purpose, write a script that selects the |L|most frequent tokens (obtained without normalization or stopword removal in poem document), then computes the ratio of the total number of tokens in L which are present in the top |L| most frequent tokens, for each poem. Summarize the result in a table and comment on the usefulness of list L. Next, we want to compare the TF-IDF
3. We want to study the layout of occurrence of stopwords in a poem. For this purpose, for each stopword occurring in poem, report the position of its occurrence in the corresponding line where the first token of the line is assigned position 1, the next token has a position 2, etc. For each stopword, one can then a generate a probability distribution over position number showing the proportion of occurrences at position 1, position 2, etc. Save these distributions in a database. Use the corresponding bar plot to show at least for the five most frequent stopwords, the distributions of their line positions. Comment on whether some heuristic observations can be statistically tested. Repeat this process for each of the poem documents.
4. We want to study the occurrence of the stopword with respect to the nature of part-of-speech tag of the preceding token (if available) and following token (if available) (represent by ∅ when unavailability occurs for either preceding token or following token). For this purpose, similarly to 3) determine the probability distribution of each stopword with respect to various part-of-speech tags, and draw the corresponding bar plot for the most frequent stopwords. Discuss to which extent can part-of-speech tag of preceding and following token can distinguish between various stopwords.
5. We want to study the occurrence of stopwords with respect to tokens occurring next and before (if available) to the given stopword in the same line of poem. Write a script that identifies for each stopword the five most frequent tokens that occur next to it and the five most frequent tokens that occur prior it (we restrict to the single token that follows / precedes the corresponding stopword.
6. We want to study the occurrence of the token that follows and precedes each stopword from the perspective of TF-IDF scoring. For this purpose, write a script that calculates for each stopword the TF-IDF scores of the token that occurs prior and next (within the same line of poem). Save the results in a database file. Use your own exploration analysis to find out whether some trends can be noticed from histogram of TF-IDF scores of previous and

following tokens. Especially, can the TF-IDF score distribution distinguish between various stopwords? Note: for the calculus of TF-IDF, each line of poem should be considered as separate document so that the document frequency is interpreted as the number of lines of poem containing the underlying token.

7. Repeat 6) when one considers the difference of TF-IDF scores between the following and previous token TF-IDF scores.
8. Now we want to combine all the attributes above (position of stopword on the poem line, part-of-speech tag of previous and following tag, TF-IDF of previous and following tag, difference between TF-IDF scores of previous and following tag) in order to build a machine learning model that predicts the occurrence of a given stopword. Suggest a data processing pipeline, feature selection and use 80/20 % split of training / testing.
9. Repeat 8) when one wants to predict the position of the stopwords one the line.
10. We want to test the extent to which a given stopword that occurred at given line n is likely to occur in the next line or the next next line. Suggest an approach that would allow you to predict the occurrence of the given stopwords in either of the two next lines.
11. Suggest your own initiative to test new hypotheses regarding the occurrence of stopwords in poem using advanced transformer architectures or LLM.
12. Identify appropriate literature to discuss your findings and comment on the limitation of the theoretical approach stated in these specifications.

Project 6: Stopword behavior in Poetry analysis 2

This project aims to explore the occurrence of stopword list in poetry. We first consider the default English stopword list in NLTK, called, say, *L*. Also, consider available poems in Gutenberg collection of open books, available from NLTK dataset. Select two poems of antagonist topics.

1. Write a program that determines the TF-IDF score of every token of each poem (without any normalization of the original text), and save the result in a database. Estimate how the TF-IDF score of stopwords compare to the rest of the TF-IDF of other tokens. It is commonly acknowledged that TF-IDF of stopwords should be low compared to rest of the non-stopword tokens. Discuss the validity of this statement using appropriate statistical hypothesis testing. Suggest appropriate visualization that would allow you to visualize the TF-IDF weights of stopwords and other tokens in the same plot. Note: the calculus of the TF-IDF of a single poem assumes a corpus of all tokens of the poem, while each like of poem is treated as a separate document to calculate the document frequency.
2. Repeat the previous specification when considering the BM25 weighting instead of TF-IDF scoring.
3. Repeat the preceding when the weighting is quantified using term-entropy.
4. We want to study the connection between the stopword occurrence and sentiment polarity. For this purpose, write a script that uses Vader sentiment analyzer to calculate the sentiment score of each line of the poem containing at least one stopword.
5. Plot the evolution of line sentiment score from beginning till the last line (containing at least one stopword), and study whether some trend can be noticed and validated using some statistical hypothesis method.
6. Now we want to study the profile of each stopword with respect to sentiment score. For this purpose, devise the range of sentiment score into 10 equally spaces bins. Then for each stopword, one calculates the number of lines containing this stopword and whose Vader score falls within a given bin, yielding a histogram distribution for each stopword. Then we want to determine the similarity between these stopwords with respect to sentiment polarity. For this purpose, for each pair of stopwords, calculate the distance between the corresponding histograms (use already established metrics for distance between histograms). Provide a graphical illustration showing the proximity of the various stopwords
7. We want to assess the semantic contribution of individual stopword to whole line of poem. For this purpose, use the following construction: For a given stopword S in Line K, use the BERT embedding to generate the embedding vector BERT(K) of the whole Line K. Then for each token T_i (including stopword S) in K, compute the cosine similarity between BERT(K) and BERT(Ti). Finally, compute the contribution of S, say, Cont(S) as the ratio

$$Cont(S) = \frac{Cosine(BERT(K), BERT(S))}{\sum_i Cosine(BERT(K), BERT(T_i))}$$

Now in order to evaluate the contributions of stopword S over all lines of the poem, we repeat the process initiated in 6) by considering the range of Cont(S) across all lines, which will then be splitted into 10 equally bins, then compute the histogram of each stopword S accordingly. Suggest an approach to identify most contributing stopword, and discuss how the stopwords are similar to each other with respect to this semantic contribution.

8. We want to study the line poem from the phonetic perspective by focusing on the sound of the last token in each line of poem. We shall consider the International Phonetic Alphabet (IPA) (shall identify relevant phonetic library for NLP). Then for each stopword and line, assign the corresponding Alphabet. Then use histogram distribution (over the 44 Alphabets of English language) and characterize the profile of each stopword accordingly as well as discuss the similarity between stopwords from phonetic perspective.
9. Use your own self initiative to go beyond the above specifications to uncover more useful patterns about stopword behavior in poem.
10. We want to reproduce specifications 2) and 3) for a 2 non-English languages. By noticing that NLTK already supports several other languages and where the default stopword lists are freely available. Select 2 other languages of your choice, and use of a part-of-speech of your choice to generate the average, standard deviation, maximum and minimum of pairs of words that have WordNet link.
11. Identify appropriate literature to discuss your findings and comment on the limitation of the theoretical approach stated in these specifications. Provide a concise and critical summary of the related literature.

Project 7: Stopword behavior in Chat corpuses 3

This project aims to explore the occurrence of stopwords in chat corpuses. We shall consider popular NPS_Chat corpora available in NLTK, and Ubuntu Dialog Corpus, ConvAI3 dataset. Initially, the below specifications are applied to NPS Chat dataset, and then are extended to two other corpora. We shall also consider the default NLTK stopword list.

1. We consider the various dialogue acts labeled in NPS Chat corpus. Group posts belonging to the same dialogue act all together, creating 15 subgroups (15 dialog acts). Write a script that identifies the various stopword in each subgroup, displays the proportion of stopwords (with respect to total number of tokens in the subgroup) and plots the corresponding histogram indicating the frequency of each stopword in the given subgroup. Study whether some stopwords are more represented in a given dialog act over others. Use appropriate statistical hypothesis testing to validate.
2. Study the extent to which the NLTK default stopword list is representative in each dialog act corpus. For this purpose, consider the set of stopwords L available in the dialog act corpus, then identify the |L| most frequent tokens in the underlying dialog act corpus, and calculate the proportion of these most frequent tokens that are also contained in L.
3. Use Spacy named-entity recognition package to identify named entities in each dialog act corpus. Draw a wordCloud representation of the set of named-entities identified by Spacy package. Study the stopwords that co-occur with named-entity (at distance one). Draw a bar plot for each dialog act corpus showing frequency of occurrence of stopword within distance one from named-entity. Discuss whether some trends can be noticed and validated through testing hypothesis.
4. We want to consider the various verb types (modal, auxiliary, action). Suggest your own approach to identify these verb types from each dialog act corpus, and then write a script that identifies the stopword that occurs within distance one (one token) from the underling verb. Then construct the histogram or bar plot showing, for each dialog act corpus, the frequency of occurrence of each stopword as modal verb co-occurrence, auxiliary verb-co-occurrence and modal verb co-occurrence. Discuss whether some trend in terms of dominance of some verb category with specific stopwords, and use statistical testing to test this trend or observation.
5. We want to study the efficiency of the categorization according to dialog act. For this purpose, apply LDA (latent Dirichlet Allocation) model for each dialogue act corpus, with parameter number of topics = 1 and number of words per topic = 10. This generates 10 keywords as a representing for each category (dialog act label). Next evaluate the mapping between the generated keywords and the category label. For this consider a simple string matching between the 10 generated keywords and label keyword, where the matching is assumed held if the label keywords is one of the 10 generated keywords. Test this matching for each of the category label.
6. Use the word2vec embedding of each LDA-generated keyword, followed by t-SNE to represent the keyword as a point in a 2D Cartesian space. Use the visual analysis of both keywords and

label in order to comment on the efficiency of the categorization and proximity with label for each dialog act corpus.

7. Instead of string matching in 5), consider a simple LLM of your choice where you input the LDA generate keywords and the LLM generates a single keyword that best describes the meaning of the set of tokens generated by LDA. Then evaluate the semantic similarity between the global keywords and the label.
8. Instead of considering only the global term generated by the LDA keywords, we next consider also the coherence of the words generated by LDA as well. Suggest an approach that uses the word embedding in order to evaluates both the semantic coherence between keywords generated by the LDA and semantic similarity with the label (e.g., taking average of cosine similarity of each keywords with label, and then see if the average is beyond a certain threshold)
9. Suggest your own initiative in order to improve the evaluation of the matching between the dialog act corpus and the corresponding label using state-of-the-art transformer architectures, and comment on the contribution of stopwords.
10. Identify relevant literature to comment on the findings and discuss the results in relevant theoretical lenses. Provide a critical summary of relevant state of the art literature in the field study.

Project 8: Stopword behavior and Corpus exploration 4

This project aims to explore the occurrence of stopword list in Old English corpus and more recent English corpus. We consider the default English stopword list in NLTK, called, say, *L*. The project utilizes two datasets: Old English corpus, available in Listing folder '/download/ORACC/2017-09/vrt/' at www.kielipankki.fi, which is manually PoS tagged, and includes Corpus of Ancient Mesopotamian Scholarship; Digital Corpus of Cuneiform Lexical Texts; Royal Inscriptions of Babylonia online; Royal Inscriptions of the Neo-Assyrian Period and State Archives of Assyria Online.

1. Write a program that extracts each of the Old English corpus: Corpus of Ancient Mesopotamian Scholarship; Digital Corpus of Cuneiform Lexical Texts; Royal Inscriptions of Babylonia online; Royal Inscriptions of the Neo-Assyrian Period and State Archives of Assyria Online and save it in an easy format for subsequent reasoning. Then provide a high level statistical information for each corpora in terms of number of tokens, vocabulary size; minimum, maximum and average number of characters per token; minimum, maximum and average length of sentence. Summarize the result in a table.
2. For each corpora, draw the term frequency of each token and test whether the Zipf's law distribution can be fit. Draw the fitting on log-log scale and 90% bounding intervals, as well as calculate the R^2 value for goodness of fit.
3. We want to test evolution of vocabulary size as a function of the number of tokens, Study to which extend the Heaps law can be fit for each corpora. Similarly to 2) draw log-log scale and 90% confidence bounds, as well as estimate the R^2 score for goodness of fit.
4. Evaluate the extent to which the NLTK stopword list is representative in the corpora. For this purpose, for each corpus, determine the K most frequent token in the corpus, where K is number of NLTK stopword list present in the corpus, then one reports the proportion of these K most frequent tokens present in the NLTK stopword list.
5. Repeat processes 2) and 3) when stopword removal task is carried out. Conclude whether the stopword removal enhances or degrades the power-of-law distribution fitting. Use statistical testing to test the claim of enhancement or degradation.
6. Consider the implementation of an information-theoretic approach for stopwords' identification from a corpus, available in Gerlach et al.'s paper A universal information theoretic approach to the identification of stopwords | Nature Machine Intelligence. Use the available implementation to determine the stopwords list for each corpus, and then repeat the task 4) to evaluate the extent to which the stopword list is compatible with the frequentist view on the token occurrence for each corpus. Calculate the TF-IDF and information content of each stopword as identified by the Gerlach et al.'s paper. Suggest relevant metrics to evaluate the usefulness of the identified stopwords that take into account the TF-IDF and information content evaluation.

7. Write a script that removes stopword list in 6) from each corpus and then re-examine the Zipfs' law and Heaps' law fitting as in specifications 2) and 3). Conclude whether the new stopword increases or decreases the power-of-law fitting performance.
8. For each keyword, one determines the part of speech tag of the token that precedes or follows the stopword (if the token does not coincide with end of sentence), then one determines the profile of each stopword by counting the frequency of each PoS tag that co-occur with this stopword, so that each stopword can generates a histogram distribution over the set of PoS tags. Use this histogram distribution to determine the similarity matrix indicating how similar the stopwords are to each other. Repeat this process for each corpus.
9. The corpora contain several out-dictionary terms, use the wordnet lexical database to evaluate the proportion of out-of-vocabulary words (terms that do not have entry in wordnet lexical database are considered as out-of-vocabulary words), and present the result in a table.
10. We want to evaluate the possible contribution of these out-of-vocabulary words in semantic meaning of sentences. Suggest an LLM based approach of your choice to evaluate such contribution. You may notice for instance that word2vec ignore out-of-vocabulary terms, while these are considered in FastText.
11. Take your own initiative to identify and implement more state-of-the-art approaches for automatic stopwords identification from corpus, and discuss potential comparison with the methods implemented above.
12. Identify relevant literature to comment on the findings and discuss the results in relevant theoretical lenses. Especially, provide a concise and critical summary of relevant literature.

Project 9: Zips Law distribution and Suomi24 Corpus.

This project investigates various facets of the Zips law and how it can be empirically tested on Finnish Suomi24 corpus.

Download the Suomi24 dataset from *The Suomi24 Corpus 2001-2017, VRT version 1.1 – META-SHARE (csc.fi)*.
You should click on the access location ***http://urn.fi/urn:nbn:fi:lb-2020021802***
Select University of Oulu in Korp and then login with your university credential to have access to the data. The data is quite large 65GB, so you should make sure that either your local computer (s) or cloud server account will be able to handle it and run basic search.

1. Select keyword "radicalization or polarization" (with all its finish potential translations) that is likely actively occurring in Suomi24. Now we want to investigate the occurrence of this keyword and related vocabulary over years. Write a script that query the whole dataset for the query terms associated to radicalization and polarization (Query 1). Save the thread titles /contents together with their associated dates in a database file of your choice.
2. Repeat the process 1) for antonym word of polarization / radicalization, e.g., moderation, integration positive citizenship, deradicalization with their Finnish translations and related vocabulary (Query 2).
3. Suggest a script that draws on the same plot the yearly evolution of the number of threats associated to Query 1, the yearly evolution of the number of threats associated to Query 2, and the yearly evolution of the total number of threats that are associated to both Query 1 and Query 2.
4. We want to elaborate more in terms of the scope of outcome to Query 1 and Query 2. For this purpose, write a script that calculates the yearly evolution of the average size (in terms of number of tokens) of threats associated to Query 1 and that associated to Query 2. Display the result in plot.
5. Repeat specification 4) when considering the size of the vocabulary in the outcomes to Query 1 and Query 2.
6. Suggest a script that would allow you to test Heap's law for Query 1 outcome, and for Query 2 outcome data. Plot the corresponding graph and estimate the statistical significance in terms of R-squared and adjusted R-squared statistics (See previous projects for some pointers).
7. Suggest a script, which identifies, for each year (from 2001 till 2017), the most frequent 10 words, excluding stopwords, in outcome of Query 1 and outcome of Query 2 data and suggest appropriate graphical illustration that also shows their intensities (number of occurrences of each of these words).
8. Write a script that compiles only the titles of the threads in each year for both Query 1 outcome and Query 2 outcome, and draw a wordcloud representation for each year (use intelligent and compacted representations of the figures to illustrate the findings in minimal plots). You may inspire from implementation in https://github.com/amueller/word_cloud, among others.
9. We would like to test the evolution of sentiment in each Query result. For this purpose, you may restrict to the top ten threats in terms of number of tokens per each year in the output result of each Query and perform the following sentiment analysis. Utilise the lexicon available in https://github.com/Helsinki-NLP/SELF-FEIL with a focus on positive and negative categories and compute the overall positive sentiment and overall negative sentiment according to the sentiment score of lexicon words available in each thread, then average the result within [0,1] interval. Suggest a script that draws the yearly evolution of positive and negative sentiment of each query result.
10. Use your own methodology to identify from news or query search, the main events that (could) impact polarization / radicalization, e.g., terrorism attacks, far-right activities, strong policy shift, and suggest an approach to correlate the findings with the Suomi24 findings.
11. Take your own initiative to propose an LLM-based approach or transformer-like approach to map Suomi24 trend to news findings.

12. Identify from your reported literature state of the art method that you can accommodate to achieve any of specification above. Motivate your answers and manipulations by appropriate literature citation.

Project 10: Zips Law distribution in food domain.

This project investigates various facets of the Zips law and how it can be empirically tested on a set of well-known corpuses.

Download the Suomi24 dataset from *The Suomi24 Corpus 2001-2017, VRT version 1.1 – META-SHARE (csc.fi)*.
You should click on the access location ***http://urn.fi/urn:nbn:fi:lb-2020021802***
Select University of Oulu in Korp and then login with your university credential to have access to the data. The data is quite large 65GB, so you should make sure that either your local computer (s) or cloud server account will be able to handle it and run basic search.

1. Consider the Fineli database of Finnish food maintained by THL, which includes over 4000 foods and their nutrition intake, see Frontpage - Fineli, for database access and detail. Write a script that search for food entries (Fineli) in Suomi24 and retrieve the threads and messages where these foods occurred together with the associated date of the threads. Now we want to investigate the occurrence of these foods over time. Write a script that gathers the foods occurred monthly and calculate the total nutrition intake by summing up the corresponding nutrition intake of each food according to Fineli database, and draw the graph of timely evolution (monthly) of the nutrition intake according to the mentioned food.
2. Write a script that gathers the thread titles associated with the identified foods in 1) in each month to form a single dataframe (for each month). Now we want to determine the sentiment polarity of each dataframe. For this purpose, use the AFINN model for sentiment analysis that employs simple lexicon-based approach, see implementation in GitHub - fnielsen/afinn: AFINN sentiment analysis in Python, to determine the overall monthly when inputting each dataframe. Draw the evolution of sentiment score and nutrition intake on the same plot, and calculate the Pearson correlation to evaluate the correlation between the two entities.
3. Repeat 2) when considering yearly evolution of sentiment and nutrition intake instead of monthly evolution. Comment on whether further refinement of the timely evolution can illustrate correlation between the two entities.
4. Now we want to explore the food diversity according to the records in 1). For this purpose, write a script that calculates the number of distinct foods in each dataframe (monthly), and then draw on the same plot the timely evolution of the food diversity and nutrition intake, and then calculates the Pearson correlation between the two entities. Repeat this process when yearly evolution is considered instead of monthly evolution.
5. The use of Fineli for quantifying food diversity may not be ideal, propose your own manual categorization based on your understand of the various types of foods mentioned in Fineli, and then re-process specification 4) accordingly.
6. Now we want to explore the asymptotic behavior of the vocabulary employed in 1). For this purpose, suggest a script that calculates the vocabulary size of thread titles gathered in month 1, in month 1 and 2, in month 1, 2 & 3, and so on. Next write a scripts that shows the evolution of the vocabulary size with respect to number of tokens in Month 1, Month 1 &2, Month 1, 2 & 3, and so on, and study the fitting of the Heap's law. Evaluate the goodness of fit using R-squared and adjusted R-squared statistics.

7. Write a script that computes the length of thread titles identified in 1) and suggest a 20 bin subdivision of the length size. Then construct the graph of the frequency of each bin with respect to the ranking, and study the fitting of Zipfs' law distribution. Provide the corresponding R-squared and adjusted R-squared statistics.
8. Suggest your own initiative to apply more advanced LLMs or transformer-based approach to study the nutrition content of food mentioned in Suomi24 search outcome. Especially, devise the prompting strategy to identify the foods liked and disliked by users, and then report the yearly evolution of the (average over total number of items) nutrition score for both liked items and disliked ones.
9. Identify from your reported literature state of the art method that you can accommodate to achieve any of specification above. Motivate your answers and manipulations by appropriate literature citation.

Project 11: Historical Finnish Word Embeddings from News

This project aims to investigate how Finnish words change their meaning over time by training and comparing word embeddings on news datasets. From Kielipankki (https://www.kielipankki.fi/aineistot/ylenews/) collect four news datasets: 2022–2024, 2021, 2019–2020 and 2011–2018.

1. Suggest a script that extracts the tokens and vocabulary of each dataset after stopword removal using the default NLTK list for Finnish language. Then use the Gensim TF-IDF vectorizer to generate the dictionary, TF-IDF score and BM25 score of each token, considering all datasets) as well as the 100 most frequent tokens (excluding Stopwords) and 100 tokens with highest TF-IDF scores, and 100 tokens with highest BM25 scores.
2. Calculate the Jaccard similarity score to quantify the similarity between the datasets from the 100 most-frequent word perspectives and 100 most highest TF-IDF scores as well as 100 most highest BM25 scores. Present the result in 4 x 4 x 4 matrix.
3. Present an illustration of the content of the most frequent words, tokens with highest TF-IDF scores, and tokens with highest BM25 scores using wordCloud illustrations for each dataset.
4. Now we want to track the change of meaning over time. For this purpose consider the word “food” or “foods”. We want the co-occurring verbs, adjective/adverb and named-entity within 3 token distance from the target word “food” or “foods”. Write a script that explores each dataset and retrieves the verb, adjective/adverb and named-entity together with the year of its occurrence. Suggest a plot that shows the yearly evolution of verbs, adjective/adverb, and named-entity (display a plot for each category and restrict the display of the five most frequent verbs / adjective/adverbs, named-entity to ease readability.
5. Now for the occurrence of each food/foods, one considers the sentiment of the underlying sentence. Use Vader to determine the sentiment polarity score, and then compute the average yearly sentiment polarity accordingly. Suggest a plot that displays the yearly evolution of food sentiment from 2011-2024.
6. Instead of restricting the target word to food or foods, we want to extent the analysis to all products, items that fall within food category. Suggest an LLM reasoning pipeline that would allow you to capture all food products and repeat the specifications 4) and 5) accordingly.
7. We want to comprehend the negative sentiment to infer a possible argument that led to negative polarity. Suggest an LLM prompting strategy that would allow you to revisit the posts where negative sentiments are recorded and identify possible reasons for it. Then suggest a plot that would display the yearly key arguments behind the negative sentiment polarity.
8. Similarly, one would like to explore the topic around the food occurrence, suggest an LLM prompting strategy that would allow you to infer the topic around the food occurrence in posts, Suggest a construction that summarize the evolution of topics on yearly basis
9. Provide a discussion of the results with concrete examples of target words and frequent words, including how their associations evolved in the Finnish news context. Comment on possible social, political or cultural reasons behind the observed changes.

Project 12: Novels's book Analysis

This project aims to investigate the structure and stylistic properties of a Novel book Jane Austen's *Pride and Prejudice* available at Gutenberg resource, accessible from NLTK library, focusing on issues of manners, morality, and marriage in 19th-century England.

1. By utilizing all textbook as input, we want to elucidate the importance of individual characters. For this purpose, suggest a script that records the number of occurrences of each character in the whole textbook. Next, we assume the importance of the characters is indicated by the number of citations each character is receiving in the text statement. Summarize the result in a table indicating the importance of the characters according to i) frequency of occurrence of characters, ii) citations count of the characters.
2. Suggest a script that gathers the text stated by each character, excluding the citation quotation, and group all text stated by a given character, as a single dataframe, and then, elucidate each dataframe by identifying its vocabulary set, vocabulary size, total number of tokens, total number of adjectives, total number of adverbs, total number of verbs, total number of nouns. Summarize in a table the above main statistical characteristics.
3. Write a script that draws the evolution of the vocabulary size with respect to the number of tokens for each dataframe, and then study the fitting of the Heap's law and its statistical significance in terms of R-squared and adjusted R-squared statistics. Comment on the *talkativity* of each character.
4. Consider the modal verbs (can, may, must, shall, will, could, might, should, would), write a script that tracks the occurrence of modal verbs in each dataframe, then plot a graph showing the evolution of the number of modal verbs employed with respect to the number of tokens, and then study the possible fitting of a polynomial interpolation cure, indicating the 90% confidence curve as well. Comment on the difference between the usage of modal verbs by each character.
5. Suggest a script that identifies the five most frequent tokens, excluding the stopwords and uncommon characters in the whole textbook, then, estimate the frequency of occurrence each of these words in each dataframe of an individual character, yielding a five dimension vector for each character. Calculate the similarity of each pair of characters using cosine similarity of the corresponding vectors. Summarize the result in a table.
6. We want to study the topical trend of each character. For this purpose, use the LDA with 5 topics and 10 keywords per topic, then generate the LDA outcome for each dataframe (character related corpus). Suggest an approach to evaluate the topical similarity between the different characters, generate the similarity matrix showing the similarity score for every pair of characters.
7. Repeat 6) when an LLM based approach of your choice is employed.
8. Suggest a script that generates embedding vector for each dataframe using BERT embedding. Then use the cosine similarity to evaluate the topical similarity between every pair of characters. Comment on the comparison between the embedding and LDA approaches.
9. Repeat 6) when empath category-based embedding https://github.com/Ejhfast/empath-client is employed. Summarize the result in a matrix and comment on the results.
10. Now we want to track individual messages by each character. For this purpose, suggest a script that uses sentiWordNet lexicon GitHub - aesuli/SentiWordNet: The SentiWordNet sentiment lexicon, where each token is assigned a positive and negative score, if no lexicon is found, it is neutral, to determine the sentiment of each character message by summing up and averaging overall sentiment scores across all the lexicon terms. Then represents each character's message as a point in a triangle-based illustration with nodes corresponding to positive, negative and neutral. Trace the plot corresponding to each character.
11. Use appropriate literature to comment on the findings. Also, identify any additional input that would allow you to further elucidate any of the preceding, and use appropriate literature of corpus linguistic

literature to justify your findings and comment on the obtained results. Finally, comment on the limitations and structural weakness of the data processing pipeline.

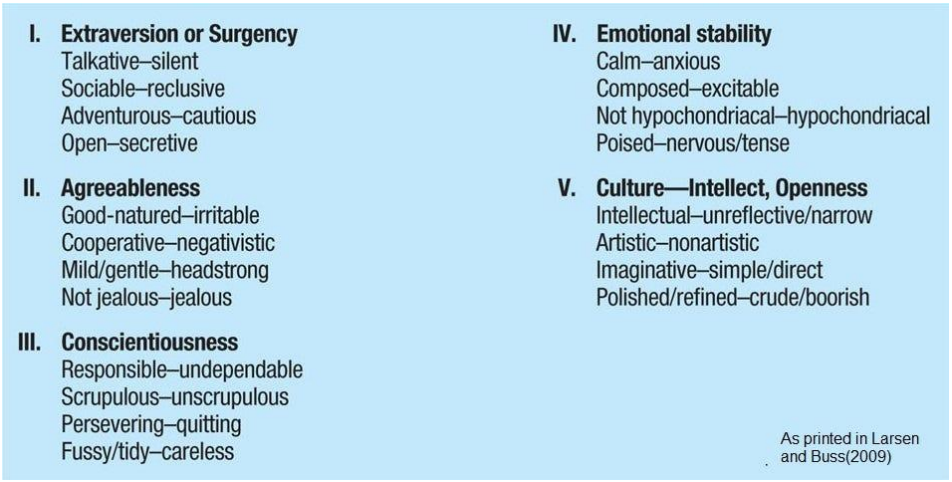
Project 13. Personality Analysis

This project explores the link between emotion and personality in dialogue systems. We especially focus on five trait personality model (extraversion, agreeableness, openness, conscientiousness, neuroticism). For this purpose, we explore the Persona-E^2 Persona-E2 Project Page , a large scale human grounded dataset mapping personality traits alongside emotional responses and appraisal shifts across news, social media and life narratives. You may study the implementation provided by the authors that use LLMs-based approach as well.

1. We want to explore the general trend of the conversation data. For this purpose, create a dataframe for each speaker by gathering all his utterances in a single file. Then write a script that determines the vocabulary set, vocabulary size, total number of tokens, total number of repetitions of words in the same post, total number of confirmation words (e.g., yes, OK, sure), total number of negation tokens, associated to each speaker. Summarize the result in a table, and then draw a subgraph that shows on the same plot the evolution of number of repetitions, number of negation, number of confirmation-like tokens, with respect to the number of tokens employed for each speaker (You may create some subdivision from the total number of tokens to ensure enough datum are used to represent the graphical illustration). Calculate the overall personality for each speaker by averaging over all instances of the original dataset, and comment on possible similarities and differences between speakers and whether some attributes are more associated with some personality patterns.
2. We want to evaluate the extent to which a speaker changes topic during conversation. In an attempt to quantify this generic trend, we hypothesize this is the case if the speaker quickly introduces new vocabulary and yields higher number of topics. To test this hypothesis, suggest a script that plots the evolution of the vocabulary size with respect to the number of tokens employed (draw on the same plot the distribution of each speaker), then comment on the suitability of the approach to discriminate key personality traits. To test the second aspect, suggest a script that uses the LDA to determine the optimal number of topics using global measures such as coherence and perplexity. You may inspire from tutoring such as Demystifying Topic Modeling Techniques in NLP | by Vijay Choubey | Medium. Similarly, comment on the suitability of the approach to highlight some personality traits.
3. We want to test whether some emotion patterns are occurring with some particular personality trait. For this purpose, we use the NRC emotion dataset where around 20K words have been assigned values for valence, arousal and dominance (VAD) dimension, see EmotionDynamics/lexicons at master · Priya22/EmotionDynamics · GitHub. Write a script that identifies NRC lexicon terms from dataframe of speaker and calculates the overage valence, arousal and dominance score normalized with respect to the total number of lexicon terms in the dataframe, as well the corresponding standard deviations, minimum and maximum value. Comment whether some valence, arousal, dominance values provide insights to discriminate some personality traits.
4. Now instead of considering overall speaker's utterances, we want to investigate individual personality trait cases. For this purpose, write a script that reports the dominant personality trait from the scores

of the five personality traits, and then constructs a new dataframe for each personality trait by gathering all utterances that fall on this category. Then repeat the process of estimating the vocabulary set, vocabulary size, number of repetitions on same post, number of confirmation-like tokens, number of negation-like tokens. Then, for each dataframe, draw a plot showing the evolution of the size of vocabulary, number of negation-like token, number of confirmation-like tokens, with respect to the total number of tokens employed (you may create a subdivision from the total number of tokens of each dataframe). Comment whether some specific discrepancy can be spotted among the various dataframes.

5. We want to consider the definitions of each personality trait given in the figure below



Suggest a script that uses an approach for generating embedding of all dataframe and another embedding for each definition term, then it uses the cosine similarity to evaluate the similarity between the dataframe and personality definition. Especially, if personality trait A has multiple definition, say, for instance T1, T2, T3. Then for dataframe embedding T, the score assigned to trait A is computed as

Score (A) = max (cosine (embedding(T1), T), cosine(embedding(T2), T), cosine(embedding(T3), T)

You may use doc2vec embedding, DistilBERT embedding

Comment on the compatibility between the embedding-based personality and the labelled dataset.

6. We want to provide an explanation for the findings. For this purpose suggest an LLM prompting strategy that would allow you to ask the model why a specific personality trait has been generated. Demonstrate your findings and provide examples of visualization for various personality traits.
7. Now we want to study the advanced implementation reported in the provided database link, use your own initiative to seek to integrate explanation of the findings and use the same performance metrics employed by the authors to demonstrate any improvement in performance.
8. Use appropriate literature to comment on the findings. Also, identify any additional input that would allow you to further elucidate any of the preceding, and use appropriate literature of corpus linguistic literature to justify your findings and comment on the obtained results. Finally, comment on the limitations and structural weakness of the data processing pipeline.

Project 14: Mining nature from the web

This project aims to assess how the search engine can teach community by promoting documents that can deliver positive message about the target word. We shall consider the target word “climate change”.

1. We want to initially analyze the outcome of Google search for a query “climate change”. Write a script that would enable you to retrieve the 100 snippets corresponding to the result of the search. You can inspire from example in https://www.thetopsites.net/article/52274742.shtml. You may notice that using free Google developer account would not allow you to reach 100, so it is fine to manually copy and past the snippets and save the result in an excel sheet. You are also free to explore snippets from other search engine that put less limitations on number of snippets gathered using free account. To increase the representativeness of the result, it is desirable to make the query without your Google login and with different login accounts, and if possible to use remote connection from another country, that would be ideal. This is become google search is very complex and often takes into account personalized aspects (e.g., profile, history of search, location).
2. Concatenate all snippets into a single document, and then draw a WordCloud representation of the document. You may see examples of WordCloud in Creating Word Clouds in Python | Engineering Education (EngEd) Program | Section
3. We shall consider each snippet as a single document, say Qi for the ith snippet. Use the WuzzyFuzzy string matching (see example in the course handout) to calculate the score of string matching of each pair of the set of the first 10 snippets of the search outcome. Especially, calculate the value of three variables: V1 corresponding to the total number of similar snippets (string matching score equals 100%); V2 for the average string matching of all pairs; V3 for the standard deviation corresponding to this average value. Now repeat this process for the first 20 snippets, then first 30 snippets, .. set of 100 snippets. Draw the plots showing the variation of V1, V2 and V3 in this consonant representation (embedded set) of the search outcome.
4. Now we would like how to see how the search outcomes differ in terms of sentiment polarity. For this purpose, use Vader sentiment analyzer to calculate the sentiment polarity of each snippet. Using previous consonant subdivision, and let V4 be the vector carrying the average sentiment for each subset (first 10 snippets, first 20, first 30, .., first 100) and let V5 be the vector carrying the associated standard deviation values. Plot the variation of V4 and V5.
5. Now we would like to investigate what are the actions, key take away message, positiveness and reference quality used in the webpage pointed out by the search engine. For this purpose, design an LLM prompting strategy where you ask the LLM to explore the web content of the page, write a short three sentences as a take-away message, summarize the action requested from the user, the perspective of climate change (positiveness versus negativeness) and comment & rank the quality of the references employed in the page, then display the outcome in concise manner.
6. Suggest an NLP data processing pipeline that allows you to visualize the evolution of the take away message, positiveness, awareness, quality of the references employed when moving from first search outcome to last search outcome.
7. Suggest a GUI that allows you to illustrate the functioning of the various specification of your project

8. Provide a concise and critical analysis of literature on climate change communication highlighting the global trend in conveying climate change aspects.

Project 15: Sentiment Analysis of Hotel Review

This project aims to investigate the sentiment and test various architectures for comprehending sentiment polarity in London hotel dataset. We shall consider the hotelRec dataset, a large scale 50 million TripAdvisor reviews with user profiles, overall ratings, titles, text, and sub-ratings. It is designed for large-scale recommendation systems and deep learning research. We shall focus on the ambiguity that may raise among the users.

1. Use the SentiStrength from http://sentistrength.wlv.ac.uk/ to determine the positive, negative and overall (sum of positive and negative) sentiment score for each review. Provide a database D1, which contains these information for each review.
2. Use Pearson correlation coefficient to evaluate the correlation of the overall sentiment score of each review with the user's rating.
3. Gather all reviews pertaining to the same hotel altogether. Calculate the average, standard deviation, kurtosis of user's ratings. Identify the hotels with low standard deviation and cases with relatively high standard deviations. Comment whether the high variation of standard deviation occurs in expensive hotel or cheap hotels (using for instance information about the hotel stars). Construct a histogram showing for each star category, the proportion of hotels for which the standard deviation is greater than some threshold ε (that you select yourself, depending on the observed range of variation of the standard deviation). We shall refer to this new class of hotels whose standard deviation is beyond the threshold as Ambiguous Class.
4. Assume that for a given hotel whose users' rating standard deviation is higher than the pre-selected threshold, if the majority of the reviews are assigned negative sentiment according to user's rating, then the hotel is assumed to belong to negative class. Likewise for positive sentiment. Provide therefore the two subclasses of this ambiguous class review. Provide an attribute in the database D1 indicating whether a given hotel belongs to Ambiguous class and whether it is positive or negative subclass. Draw a histogram highlighting the proportion of positive and negative subclass in the original Ambiguous Class.
5. We would like to explore the content of each of the above subclass. Concatenate all reviews of each subclass and use wordCloud (see an example at https://www.geeksforgeeks.org/generating-word-cloud-python/) that highlights the most frequent wording used. Plot WordCloud for both positive and negative subclass. Comment on the findings
6. Use LDA with five topics and five words per topic to determine the topic distribution of the positive and negative subclass. Report the result in the database 1. Compare the result of the LDA output with the WordCloud finding (check the extent of the overlapping with high frequent words for instance or whether some LDA wording are directly related to WordCloud via other semantic network.
7. Consider categories generated by Empath Client https://github.com/Ejhfast/empath-client. Apply Empath Client to positive and negative subclasses reporting only those categories who held non-zero values. Record these findings in database D1.
8. Calculate the ratio of overlapping over the number of Empath categories as an indication of agreement between empaths and common corpus.
9. Repeat 8) by calculating the agreement between Empath categories and LDA.
10. Consider the standard five class categories often employed in hotel review: Price, Service, Parking, Room, Location, Food. Suggest your own ontology construction that gathers words associated to each category (for instance using list of synonym, thesaurus, hypernyms,…).

11. Use your developed ontology to seek the histogram of the frequency of the occurring five categories for both positive subclass and negative subclass.
12. From Huggingface transformers library, identify relevant LLM model for sentiment analysis, and use your own prompting strategy to evaluate the performance of the sentiment analysis. Report the performance metrics results and compare it with previously employed sentiment analyzers.
13. We want to focus on bias in user reviews. For this purpose, identify instances where the sentiment score is very negative while user's rating is high, and similarly cases where test review is very positive while the user rating is very negative. Suggest your own NLP-based methodology to tackle this issue and report on possible causes for this bias.
14. Design a simple GUI of your choice that shows the execution of each of the above tasks in a way to ease the task of the assessor or external end-user.
15. Provide a concise and critical summary of relevant literature in hotel review analysis, and provide a basis for commenting and discussing the results found in this study.

Project 16: CSWO-Bench: A Multilingual Multimodal Benchmark of Citizen Science Water Observations (Proposed by Moinul Islam)

Citizen science platforms across Europe and North America collect water observations from volunteers, but each platform stores them in its own schema, language and licence, and the free-text part of these records (observer notes, comments, descriptions) has never been assembled into a corpus for NLP research. This project builds and documents that corpus. Students harvest observation records that carry text, photos and structured labels (water level class, Forel-Ule colour index, Secchi depth, turbidity, species) from at least four platforms, harmonize them into a single schema, and run a standard corpus analysis and a set of baseline tasks on the result. The target is a dataset and benchmark package of the kind submitted to the NeurIPS Datasets and Benchmarks track: fixed splits, baselines, a data card and a licence audit.

The closest prior work uses social media rather than citizen science records (the MediaEval 2021 WaterMM task on water quality in tweets, and a 2024 LLM framework for water-related tweets). Students should cite these and show where the new corpus differs.

Each record is stored in a common JSON schema with at least the fields: id, source platform, language, original text, English translation, media file names or URLs, timestamp, coarsened coordinates, structured labels, licence, and provenance (original record URL). Expected scale is tens of thousands of text records, not millions. Languages reachable through the sources below are English, German, French, Swedish, Finnish, Estonian and Latvian.

Data sources

Use at least **four of the platforms** stated here, including at least one text-rich source and at least one Baltic or Nordic portal.

Specifications

1. Write scripts that collect observation records from at least four of the platforms listed above using their public APIs, bulk downloads or GBIF. For GBIF, request a Darwin Core Archive filtered to the chosen countries, aquatic taxa and non-empty occurrenceRemarks, and record the download DOI. Keep only records with a licence that permits redistribution. Store the licence and the original record URL with

every record. Filter biodiversity portals to aquatic taxa and water-body localities so that every record concerns a lake, river, stream, coast or sea.

2. Harmonize all records into the common JSON schema. Run language identification on every text field. Translate non-English text to English with an open model (for example NLLB or a Helsinki-NLP OPUS model) and store both versions. Coarsen coordinates to 1 km and remove or hash user identifiers. Summarize in a table the number of records per platform and language, and the share with text only, image only, and both.
3. Compute corpus statistics per platform and per language: total tokens, vocabulary size, mean and standard deviation of note length, and the proportion of tokens with a WordNet entry (English) or a FinnWordNet or OMorFi entry (Finnish). Compare these figures against a standard corpus such as Brown or Reuters and comment on how short, noisy field notes differ from edited text.
4. Test Zipf's law per platform and per language on the log-log rank frequency plot with a linear fit and 90% confidence bounds, and report R-squared and adjusted R-squared. Test Heaps' law by drawing vocabulary size against the number of tokens at 5% increments. Comment on whether the laws hold for crowdsourced micro-text.
5. Use spaCy and a multilingual transformer NER model to extract named entities from the text fields, and add rule-based extraction for water condition terms (algae, foam, ice, turbid, clear, smell, oil, flood, dry).
6. Build a gold standard. Each student annotates 100 records drawn from a shared stratified sample, so that a group of n students produces an n x 100 record gold set. Every record is labelled by two students. Labels are: water condition (multi-label: clear, turbid, coloured, algae, foam, ice, oil, litter, odour, high water, low or dry, flooding, none stated), entity spans (WATER_BODY, LOCATION, ORGANISM, QUANTITY, DATE_TIME, WEATHER), relevance to water (yes, no, uncertain), and, where a photo exists, image-text consistency (consistent, partly, inconsistent).
7. An annotation guideline with definitions, examples and rulings for hard cases is provided here. Read it, run the 20-record calibration round it describes, add every resolved disagreement to its hard-cases table, then report Cohen's kappa for record-level labels and span F1 between annotators, and resolve disagreements by discussion. Read the original-language text if you can. Otherwise read the machine translation shown beside it and record which you used. Mark entity spans only on records read in the original. Then report precision, recall and F1 for each extractor on the gold set, and compare English-original text against machine-translated text.
8. Use the structured fields (CrowdWater water level class, EyeOnWater Forel-Ule index binned into three classes, Artportalen habitat type) as weak labels. Train TF-IDF with SVM and a fine-tuned multilingual DistilBERT to predict these labels from text alone, with an 80/10/10 split stratified by platform. Report precision, recall and F1, and repeat with and without stopword removal.
9. Measure text-image consistency. Compute CLIP or SigLIP embeddings for each photo and its note, score the cosine similarity, and check the score against the image-text consistency labels from the gold set built in 5. Report the correlation and comment on which platforms produce the most consistent pairs.
10. Run a zero-shot LLM and a vision-language model (VLM) on the held-out gold subsets from 5 and 7 and compare with the supervised baselines. Define hallucination as any claim in the output not supported by the note or the image, count it on 100 outputs per model, and report the hallucination rate.
11. Audit bias in the corpus: spatial coverage by country on a map, language imbalance, seasonal distribution, and per-user contribution skew measured with the Gini coefficient. Discuss how each affects the benchmark results in Steps 5 to 8.

12. Package the corpus with fixed train, validation and test splits, the baseline scripts, and a data card following the Datasheets for Datasets template. The data card must list sources, licences, anonymization steps, known limitations and a recommended citation. Release media as URLs where the platform licence does not permit redistribution of files.
13. Take your own initiative and identify one state-of-the-art approach for multilingual or multimodal environmental text mining that can be run on the corpus and compare it with the baselines above.
14. Identify relevant literature in citizen science, corpus linguistics and dataset documentation to discuss the findings, and comment on the limitations and structural weakness of the data processing pipeline (when writing the report).

Deliverables

Deliverables are,

- the harmonized corpus in JSONL with splits,
- the collection and baseline scripts in a public repository,
- the data card, and
- a report that presents the tables and plots from specifications 2 to 9 with the discussion from 11 and 14.

Notes for Students

- Start with EyeOnWater and iNaturalist, which have the simplest open access, then add CrowdWater and one Baltic or Nordic portal. Keep a licence log from the first download.
- The gold set in specification 5 is the most valuable output of the project. Use a shared annotation tool (Label Studio or INCEpTION) and export it with anonymized annotator IDs so that it can be released with the corpus. Coordinates and usernames are personal data and must be coarsened or removed before any sharing.
- For Finnish text, the Turku neural parser, FinnPos or OMorFi listed under the course resources can replace NLTK tools.

Project 17: Explainable Food Recommendation, Explanation Quality Analysis & Enhancement. (Proposed by Amir Mollazadeh)

Goal

Analyze and improve the quality of natural language explanations generated by a hybrid food recommendation system. Your task is evaluating whether explanations are factually accurate, clear, transparent in internal reasoning, and free from demographic bias. You will perform NLP-based analysis on existing explanations, then generate and compare explanations from multiple LLMs to see which produces higher-quality outputs.

Dataset

The XFoodRec project has already generated:

- **`personas.json`**: ~40 synthetic user profiles with dietary goals, allergies, preferences, demographics
- **`recommendations.json`**: Generated recommendations with explanations for each persona (~15 recipes per persona = ~600 total recommendation sets). [Generated by GPT-4o-mini]
- **`recipes.parquet`**: Source recipe data (nutrition, ingredients, tags,…)

[XFoodRec GitHub: https://github.com/Amir-Mol/XFoodRec]

Specifications

1. Data Exploration & Preparation

- Load and inspect **`personas.json`**, **`recommendations.json`**, and **`recipes.parquet`**
- Extract all explanations and compute basic statistics:
  - Length (words/characters)
  - Distribution of explanation types (ingredient-, nutrition-, taste-focused)
  - Coverage of user preferences vs. health justification

2. Entity Extraction, Fact Verification & Hallucination Detection

- Extract three types of claims from each explanation: 1. Ingredient mentions (e.g., "garlic", "salmon"), 2. Nutrient claims, qualitative or numeric (e.g., "high in protein", "15g protein"), 3. Diet/health-label claims (e.g., "gluten-free", "diabetes-friendly", "low sodium")
- Compare and verify extracted claims with the actual data:
  - Ingredients → recipe's ingredient list
  - Nutrient claims → recipe's nutrition fields
  - Diet/health-label claims → both the recipe's ingredients and the persona's restrictions/allergies/conditions
- Define hallucination types:
  - Ingredient: mentions an ingredient not in the recipe
  - Nutrition: claimed value significantly differs from actual data (numeric) or contradicts the documented threshold (qualitative)
  - Diet/allergen: claims a diet/health label or compatibility with a persona restriction that the recipe's ingredients contradict (or vice versa)
- Validate the verifier itself: Manually label a sample of explanations as hallucinated/not.
- Report:
  - Hallucination rate (% of explanations with ≥1 false claim), broken down by type
  - Patterns (do longer explanations or certain persona types hallucinate more?)

3. Transparency & Reasoning Analysis

The XFoodRec prompt requires explanations to explicitly cite the user factors (goal, preference) that drove the recommendation

- Classify each explanation by transparency level:
  - High: explicitly mentions ≥1 user factor, e.g., "matches your muscle gain goal"
  - Medium: implies user relevance but not explicit
  - Low: generic, no user context
- - Report the distribution of levels and compare across persona types.
  Note: You can classify with rule/lexicon matching (a list of factor keywords + user-directed cue words such as "your"/"goal"/"needs", excluding words that only appear in the recipe title)

4. Bias & Fairness Analysis

- Investigate whether explanation quality varies by persona characteristics:
  - Dietary goal
  - Health status
  - Dietary restriction
  - Demographics (age, gender)
- Metrics:
  - Average explanation length
  - Transparency score
  - Hallucination rate
- Apply statistical tests (t-tests, ANOVA) on persona-level values with multiple-testing correction, and frame results as exploratory.
  Note: The explanations are nested in 40 personas, so it is good to aggregate metrics per persona before testing.

7. Multi-Model Explanation Generation

- Use at least 2 different LLMs (you can use Lehmus AI platform or other services available for University of Oulu students) to generate explanations for the exact same persona + recipe + rank triples already in `recommendations.json`
  Note: choose LLMs that differ meaningfully in size/architecture for a more informative comparison.
- Run each model's explanations through the Task 1-4 pipeline for comparison against the original GPT-4o-mini explanations.

9. Evaluation & Comparison

- Compare original (GPT-4o) explanations vs. new models output
- Metrics:
  - Factual accuracy (% of verified claims)
  - Transparency score
  - Hallucination rate
  - Average length
- Report a model-comparison table and discuss which model(s) produced better explanations and why .

Deliverables

- Code/notebook implementing all specifications above.
- The raw explanation outputs collected from each additional model.

- A written report covering all analyses, with tables/plots, the model comparison table, the model comparison table, and the hallucination and bias findings.

Project 18: Multilingual Clinical Text Summarization **(proposed by Aleka Ayalew)**

1. Objective

The objective of this project is to investigate how diverse NLP methods generate concise summaries from clinical text across multiple languages. You will implement and compare classical extractive summarization, multilingual transformer models, and instruction-tuned LLMs. The project evaluates how summarization quality varies across native benchmark languages (English and Spanish or French) and a translated test set (Finnish), examining document lengths, compression rates, and model hallucinations.

2. Dataset Specification

You will use the open-access MultiClinSum benchmark:

- **Dataset Link:** https://zenodo.org/records/17341582
- (Alternative : https://zenodo.org/records/15459174)
- **Content:** Clinical case reports paired with human-written reference summaries in English (`en`), Spanish (`es`), French (`fr`), and Portuguese (`pt`).
- **Language Scope:** You will evaluate on three languages:
    1. English (`en`) directly from the benchmark.
    2. One Romance language directly from the benchmark: Spanish (`es`) or French (`fr`).
    3. Finnish (`fi`) translated set: Generated by translating the English clinical cases and reference summaries into Finnish (using `Helsinki-NLP/opus-mt-en-fi` or an open instruction LLM), clearly labeled as translated data.

3. Summary Output Length and Parameter Specifications

- **Extractive Summaries:** Recommended target of 20% to 25% of source sentences (or top 4–6 sentences). You can adjust this target as you want to get better results.
- **Transformer Generation Parameters:**
    - Maximum token limit: `max_new_tokens = 512`.
    - Recommended baseline starting values: `num_beams = 4`, `length_penalty = 1.0 to 1.2`, `no_repeat_ngram_size = 3`.
    - **Parameter Adjustment:** You are not restricted to fixed values. You can adjust and tune parameters as you want (such as beam search, length penalty, repetition penalty, or sampling settings) to get the best possible summary quality and prevent text from repeating or cutting off. Keep chosen settings consistent when comparing across languages.

- **LLM Generation Parameters:**
    - Recommended starting settings: `max_new_tokens = 512`, `temperature = 0.1 to 0.3` (or greedy decoding).
    - **Parameter Adjustment:** You can adjust prompt phrasing, temperature, top-p, and length instructions as you want to get better results.
    - Baseline prompt instruction: *"Provide a comprehensive clinical summary in 4 to 6 concise sentences (approximately 150–250 words) covering chief complaint, key findings, hospital course, and discharge plan."*

4. Step-by-Step Activities

Step 1: Data Acquisition, Cleaning & Cross-Lingual Translation Pipeline

Step 2: Classical Extractive Baselines (Luhn, TF-IDF, LSA, TextRank)

Step 3: Multilingual Transformer Summarization (mT5 / mBART)

Step 4: Instruction-Tuned LLM Summarization (Zero-Shot & Few-Shot)

Step 5: Automated Multilingual Evaluation (ROUGE, METEOR, BERTScore, ChrF++)

Step 6: Translation Quality & Semantic Drift Audit

Step 7: Factuality, Hallucination & Negation Audit

Step 8: Results Visualization (Charts & Figures)

Step 9: Comparative Synthesis & Final Reporting

Step 1: Download, Clean, and Prepare Data

1. Download the MultiClinSum archive and open the language folders (`en`, `es`, `fr`, `pt`).
2. Clean the data: remove empty rows or corrupted text files.
3. Split documents into sentences and words using an NLP library like Stanza, SpaCy, or NLTK.
4. Create the test sets: select 100 documents for English and 100 documents for the chosen Romance language (Spanish or French).
5. Create the Finnish test set: Translate the 100 English cases and their reference summaries into Finnish using `Helsinki-NLP/opus-mt-en-fi` or an instruction LLM.
6. Calculate basic dataset statistics: total document count, average sentence count, average word count, and reference summary word count across all three languages.

Step 2: Implement Classical Extractive Baselines

Implement four classical extractive algorithms in Python (using libraries like `sumy`, `nltk`, or `scikit-learn`):

1. **Luhn Algorithm:** Score sentences based on the frequency of important non-stopword words.
2. **TF-IDF Summarizer:** Score sentences by calculating the average TF-IDF value of their words.

3. **Latent Semantic Analysis (LSA):** Rank sentences that best capture the main topics using matrix decomposition from standard libraries.
4. **TextRank:** Build a graph of sentences connected by shared words and rank them using the PageRank algorithm.
5. Standardize summary length across all four algorithms (e.g., top 4–6 sentences or 20% to 25% of the text; adjust this target as you want to get better summary quality).
6. Run all four baselines on the 100 documents across English, Romance, and Finnish splits, and save the outputs.

Step 3: Run Multilingual Transformer Models

1. Load a multilingual model fine-tuned for summarization (such as `csebuetnlp/mT5_multilingual_XLSum` and an **mBART** summarization checkpoint) in Python using Hugging Face `transformers`.
2. Configure language tags or task prefixes as required by the selected model.
3. Set generation settings (recommended baseline: `max_new_tokens = 512`, `num_beams = 4`, `length_penalty = 1.0 to 1.2`). You can adjust and tune these settings as you want to get better results.
4. Generate summaries for the 100 test cases per language. Record the running time (seconds per document) and the summary lengths.

Step 4: Run Instruction-Tuned LLMs

1. Load an instruction-tuned open LLM (such as Llama-3-8B-Instruct, Mistral-7B-Instruct, or Qwen2.5-7B-Instruct) using 4-bit/8-bit quantization on a local GPU or via an open-weights API with `max_new_tokens = 512`.
2. Test two prompt styles across all three languages:
    - **Zero-Shot:** Direct prompt instructing the model to act as a clinical assistant and summarize the note into 4–6 sentences (150–250 words) covering key clinical categories.
    - **Few-Shot:** Provide 2 complete example pairs (a source note + its reference summary in that language) inside the prompt before giving the test note.
3. You can adjust the prompts and temperature settings as you want to get better results. Make sure outputs contain only the summary without conversational filler. Save all generated summaries.

Step 5: Calculate Automatic Evaluation Metrics

1. Compute word-overlap metrics comparing generated summaries to reference summaries:
    - ROUGE-1, ROUGE-2, and ROUGE-L (F1 scores).
    - METEOR score.
2. Compute semantic similarity using BERTScore (F1, Precision, Recall) with a multilingual model checkpoint (`bert-base-multilingual-cased`).
3. Compute ChrF++ scores (especially helpful for Finnish to properly evaluate compound words and word endings).
4. Report the average score and standard deviation for all 7 systems (4 extractive, 1 transformer, 2 LLM prompts) across all three languages in a summary table.

Step 6: Check Translation Quality and Meaning Shifts

1. Pick a sample of 10 English-to-Finnish translated cases.
2. Back-Translation Check: Translate the Finnish text back into English and check whether the medical facts stayed accurate.
3. Check for medical translation mistakes: Look for errors in anatomical terms, medication doses, drug names, or negation (for example, whether "no fever" was accidentally translated as "had fever").
4. Document in the final report how translation errors in synthetic Finnish references affect automatic ROUGE and BERTScore evaluations.

Step 7: Manual Audit of Clinical Facts and Hallucinations

1. Manually inspect 10–15 clinical cases across models to check factual correctness across 6 core areas:
    - Primary Diagnosis / Main Problem
    - Symptoms and Relevant History
    - Key Lab and Test Results
    - Treatments and Procedures
    - Medications and Doses
    - Discharge and Follow-Up Plans
2. Count and record:
    - Hallucinations (facts invented by the model that are not in the source text).
    - Inverted facts / Negation errors (e.g., "no signs of infection" turned into "infection present").
    - Compare whether models made more mistakes in translated Finnish than in English or the Romance language.

Step 8: Create Visualizations (Plots and Charts)

Create clear figures displaying experimental outcomes:

1. **Grouped Bar Charts:** Compare ROUGE-1, ROUGE-2, and ROUGE-L across all 7 models, grouped by language (English vs. Romance vs. Finnish).
2. **Grouped Bar Chart:** Compare BERTScore F1 and ChrF++ scores across all models and languages.
3. **Box Plots:** Show the distribution of document-level ROUGE-L and BERTScore values to show model consistency.
4. **Horizontal Bar Chart:** Compare average compression ratios across models.
5. **Scatter Plot:** Plot source document length (x-axis) against BERTScore / ROUGE-L (y-axis) to see if longer texts reduce summary quality.
6. **Execution Time Bar Chart:** Compare average running time (seconds per document) across models and languages.

Step 9: Report Findings and Limitations

1. Discuss trade-offs: evaluate whether LLMs provide enough quality improvement over classical extractive baselines to justify their higher compute time.
2. Explain metric limitations: discuss why extractive models can achieve decent ROUGE scores while lacking smooth clinical readability.
3. Review related research on multilingual clinical NLP and LLM safety.
4. Document project limitations (machine translation noise, evaluation metric limitations, and hardware constraints).

Project 19: Multi-Agent Clinical Summarization on Synthetic Notes

(Proposed by Aleka Ayalew)

1. Objective

The objective of this project is to evaluate whether a multi-agent LLM pipeline improves factual accuracy, reduces clinical hallucinations, and ensures grounded summaries compared to single-agent prompts. The system divides work into distinct agents (extraction, drafting, fact-checking, and revision) that use the source clinical note as the ground truth.

2. Dataset Specification

You will use the open-access Asclepius clinical notes benchmark:

- **Dataset Link:** https://huggingface.co/datasets/starmpcc/Asclepius-Synthetic-Clinical-Notes
- **Filtering Rule:** Filter rows where `task == "Summarization"`.
- **Field Roles:**
  - `note`: Synthetic clinical note provided in the dataset (input to models).
  - `answer`: Reference summary provided in the dataset (used as the ground truth benchmark for evaluation).

3. Summary Output Length and Parameter Specifications

- **Target Summary Length:** Recommended 200–350 words (approximately 3 to 4 clinical paragraphs structured by: Hospital Course, Key Diagnoses and Procedures, Discharge Medications, Follow-up).
- **Token Bounds:** Set `max_new_tokens = 512` for all generation, verification, and revision agents.
- **Decoding Parameters & Flexibility:** Recommended starting baseline is low temperature (`temperature = 0.1 to 0.3` or greedy decoding) to reduce hallucinations. You can adjust hyperparameters and prompt constraints as you want to get better results, provided you maintain comparable target length across baseline and multi-agent summaries to ensure fair evaluation.

4. Step-by-Step Activities

Step 1: Dataset Extraction, Filtering & Clinical Schema Mapping

Step 2: Build the Information Extraction Agent (Agent 1)

Step 3: Implement Single-Agent Baseline

Step 4: Construct Intra-Document Evidence Retrieval Index

Step 5: Assemble Multi-Agent Workflow (Generation, Verification & Revision)

Step 6: Pipeline Execution & Intermediate State Tracing

Step 7: Automated Evaluation (ROUGE, BERTScore, METEOR)

Step 8: Manual Verification & Factuality Audit

Step 9: Agent Ablation & Resource Analysis

Step 10: Results Visualization (Charts & Figures)

Step 11: Synthesis, Literature Review & Final Reporting

Step 1: Load and Filter the Dataset

1. Load Asclepius and filter rows where `task == "Summarization"`.
2. Remove any rows with missing text, null values, or duplicate entries.
3. Select an evaluation set of 100 clean synthetic clinical notes.
4. Identify key clinical information to extract from each note:
    - Diagnoses and Conditions
    - Hospital Course and Patient History
    - Key Lab and Test Results
    - Treatments and Surgeries
    - Discharge Medications and Instructions
5. Calculate basic dataset statistics: average note word count, reference summary length, and vocabulary size.

Step 2: Build the Information Extraction Agent (Agent 1)

1. Write a prompt instructing the LLM to extract key patient details into a clean JSON structure:

```
{
  "primary_diagnoses": [],
  "secondary_conditions": [],
  "critical_lab_abnormalities": [],
  "procedures_performed": [],
  "medications_discharge": [],
  "follow_up_instructions": ""
}
```

2. Set `max_new_tokens = 512`. Use `json.loads()` in Python to make sure the output is valid JSON. You can adjust prompt instructions as you want to extract cleaner and more complete entities. Save the extracted JSON for all 100 cases.

Step 3: Build the Single-Agent Baseline

1. Write a single zero-shot prompt giving the full clinical note and asking for an organized clinical summary (200–350 words).
2. Run this prompt on the 100 evaluation notes using an instruction-tuned LLM (`max_new_tokens = 512`). You can adjust prompt wording and decoding parameters as you want to get better results.
3. Save these baseline summaries to compare against the multi-agent system.

Step 4: Build the Search Index for Source Notes

1. Divide each clinical note into text chunks of 250–350 tokens with a 50-token overlap between adjacent chunks (or split by section headers like CHIEF COMPLAINT, HOSPITAL COURSE, LABS). You can adjust chunk sizes and overlap as you want to get better retrieval results.
2. Index these chunks locally using BM25 (`rank-bm25`) or a sentence-transformer embedding model (`sentence-transformers/all-MiniLM-L6-v2`).
3. Write a helper function in Python: given a statement from a summary, search the source note and return the top-2 most relevant text chunks as evidence.

Step 5: Build the Multi-Agent Workflow

Connect the remaining agents in Python:

1. **Drafting Agent (Agent 2):** Takes the extracted JSON from Agent 1 (or the raw note) and writes an initial clinical summary (`max_new_tokens = 512`).
2. **Verification Agent (Agent 3):**
    - Breaks the draft summary into individual factual statements.
    - Expands common medical abbreviations using a small dictionary (e.g., "CHF" by "congestive heart failure").
    - Searches the source note for supporting chunks for each statement and labels each statement:
        - `[Supported]`
        - `[Unsupported / Hallucinated]`
        - `[Contradictory / Inverted Fact]`
        - `[Omitted Critical Finding]`
3. **Revision Agent (Agent 4):** Reads the initial draft and the verification report, then rewrites or removes unsupported statements while keeping validated facts intact (`max_new_tokens = 512`).
4. **Parameter & Workflow Tuning:** You can adjust agent prompt instructions, temperature, and verification rules as you want to achieve higher factuality and more coherent summaries.

Step 6: Run the Full Multi-Agent Pipeline

1. Run all 100 evaluation notes through the complete sequence: Extraction -> Drafting -> Verification -> Revision.
2. Save full JSON logs for every sample containing:
    - Note ID
    - Baseline single-agent summary
    - Initial draft summary
    - List of statements with their verification labels
    - Final revised summary

Step 7: Calculate Automatic Evaluation Metrics

1. Calculate evaluation scores against the dataset's `answer` column for:
    - Single-Agent Baseline
    - Initial Draft (before revision)
    - Final Revised Summary
2. Compute ROUGE-1, ROUGE-2, ROUGE-L, METEOR, and BERTScore (F1).

3. Record summary word counts and compression ratios to confirm that the revision agent does not artificially inflate scores by making summaries overly long.

Step 8: Manual Audit of Verification Accuracy

1. Manually check a sample of 15 cases across drafts, verification logs, and revised summaries.
2. Evaluate Verification Accuracy:
    - Did the verification agent correctly catch real errors, or did it falsely flag correct statements?
    - Did any hallucinations slip past the verifier undetected?
3. Evaluate Revision Quality: Check whether the revision agent fixed flagged errors or accidentally introduced new mistakes.

Step 9: Component Comparison (Ablation Test) and Resource Measurement

1. Run an ablation test on 20 notes comparing:
    - Full pipeline (Extraction -> Drafting -> Verification -> Revision)
    - Pipeline without Extraction (Drafting directly from raw note -> Verification -> Revision)
2. Measure compute usage:
    - Total input and output tokens used across agents.
    - Total running time (seconds per document).
3. Discuss whether the multi-agent pipeline produces enough improvement in factual accuracy to justify the higher token and runtime cost.

Step 10: Create Visualizations (Plots and Charts)

Create clear figures displaying experimental outcomes:

1. **Pipeline Diagram:** A clean diagram showing the flow of data between agents.
2. **Grouped Bar Charts:** Compare Baseline vs. Initial Draft vs. Final Revised Summary across ROUGE-1, ROUGE-2, ROUGE-L, and BERTScore F1.
3. **Stacked Bar Chart:** Total count of verification flags across the 100 cases categorized by error type (Hallucination, Dosage Mismatch, Omission, Contradiction).
4. **Before-and-After Bar Chart:** Number of factual errors in the initial draft versus remaining errors after revision.
5. **Scatter Plot:** Number of statements checked (x-axis) against verification running time (y-axis).
6. **Cost Comparison Chart:** Compare average runtime (seconds) and token counts (Single-Agent Baseline vs. Multi-Agent Pipeline).

Step 11: Report Findings and Limitations

1. Discuss failure modes: show specific examples where the verification agent missed subtle medical errors.
2. Relate experimental findings to recent research on LLM self-correction, verification methods, and clinical safety.
3. Document project limitations (reliance on synthetic reference data, compute runtime bottlenecks, and potential error propagation across agents).

Project 20: Lexical Substitution and Meaning Preservation in LLM-Based Text Simplification (Reserved Sajjad Ghaeminejad)

The aim of this project is to analyze how LLMs simplify text, with a particular focus on their **lexical substitution behavior**. The project investigates whether LLMs replace complex words with alternatives that are more frequent, less ambiguous, or more general in the WordNet hierarchy, and whether these substitutions improve readability while preserving the original meaning.

Dataset. Use the **ASSET dataset**[1] (Alva-Manchego et al., ACL 2020), which contains **2,359 original sentences**, each accompanied by **10 human-written simplifications**. ASSET is available through the EASSE text-simplification evaluation package. The **TurkCorpus** dataset can be used to examine generalization.

Specifications

1. Corpus Exploration and Preprocessing

- Report number of sentences, tokens, vocabulary size, average sentence length, and type-token ratio.
- Analyze the Penn Treebank part-of-speech distribution for original and human-simplified sentences.
- Visualize frequent words and content words using bar charts or word clouds.
- Tokenize, lemmatize, and POS-tag all source sentences, human references, and system outputs.

2. LLM Simplification Generation and Baselines

- Generate simplifications using **three LLM systems** with the same simplification instruction.
- Keep prompts, model versions, temperature, decoding parameters, and random seeds documented.
- Include: An identity baseline, in which the original sentence is returned unchanged. Human reference simplifications as a comparison benchmark.
- Store all generated outputs and model configurations to ensure reproducibility.

3. Readability Analysis

- Calculate readability before and after simplification using: Flesch Reading Ease, Gunning Fog Index, SMOG and FORECAST or another short-text readability measure
- Compare the original sentences, LLM outputs, and human references.
- Report readability gain for each system and use paired statistical tests and effect sizes to determine whether the improvements are significant.

4. Word Alignment and Substitution Extraction

- Align words between each original sentence and its simplification.
- Identify: Word substitutions, Deletions, Insertions and Phrase-level rewrites
- Focus the main lexical analysis on substitutions involving nouns, verbs, adjectives, and adverbs.
- Report alignment coverage and manually inspect a sample of alignment errors.
- If the main alignment method covers less than 80% of candidate substitutions, use lemma-level one-to-one alignment as a fallback and report the resulting coverage loss.

5. Lexical Substitution Profile

For each aligned source–replacement pair, calculate changes in:

- Word frequency or Zipf frequency
- Word length and number of syllables
- Number of WordNet senses as a measure of polysemy
- WordNet hypernym depth as an approximation of semantic specificity
- Semantic relationship between the original and replacement words, such as synonym, hypernym, hyponym, or unrelated word

Compare these characteristics across the three LLM systems and human simplifications. Analyze whether LLMs systematically prefer more frequent, shorter, less ambiguous, or more general words.

6. Vocabulary Distribution and Power-Law Analysis

- Fit **Zipf's law** to the word-frequency distribution of each system.
- Fit **Heaps' law** to vocabulary growth.
- Report fitted parameters, R^2, adjusted R^2, and 90% confidence intervals.
- Examine whether LLM simplifications have less lexical diversity than human simplifications.

[1] https://github.com/facebookresearch/asset

7. Compression and Content Retention

- Calculate sentence compression ratio, token deletion rate, and average sentence-length reduction.
- Measure retention of: Named entities, Numbers and dates, Negations and Important nouns and verbs
- Compare systems in terms of how much content they remove during simplification.
- Analyze examples where improved readability results from deleting important information.

8. Simplification and Meaning-Preservation Evaluation

Evaluate each system using: **SARI** against the ten human references, **BLEU** as a reference-similarity measure, **Sentence-embedding cosine similarity** between the original and simplified sentences, and **BERTScore**
Include the identity baseline and a leave-one-reference-out human benchmark. Reproduce and verify the preliminary identity and human-reference scores within the final experimental pipeline.

9. Readability–Meaning Trade-off Analysis

- Define readability gain as the difference between source and simplified-sentence readability.
- Define meaning loss using embedding similarity, BERTScore, and content-retention measures.
- Calculate Pearson and Spearman correlations between readability gain and meaning loss.
- Use regression analysis to determine whether changes in frequency, polysemy, hypernym depth, compression, and word length predict meaning loss.
- Report regression coefficients, confidence intervals, and goodness-of-fit measures.
- Create a plot showing systems that achieve best balance between simplicity and meaning preservation.

10. System Identification

Investigate whether the generated simplifications contain recognizable model-specific patterns.

- Predict which LLM produced each simplification using: TF-IDF with Logistic Regression, SVM, and Fine-tuned DistilBERT or a frozen transformer-embedding classifier
- Include a majority-class baseline.
- Report Accuracy, Precision, Recall, macro-F1, confusion matrices, and processing time

11. Interface, Error Analysis, and Documentation

- Develop a simple interface using Streamlit or Gradio.
    - Allow the user to enter a sentence and view the results
- Include representative examples of successful simplifications, unnecessary substitutions, meaning changes, entity loss, and alignment failures.
- Discuss limitations related to word alignment, WordNet coverage, word-sense ambiguity, metric reliability, prompt sensitivity, and the subjectivity of simplification.
- Provide a repository containing the code, data or data-loading instructions, generated outputs, evaluation scripts, and a README explaining which file produces each table and figure.

Extensions (Optional)

- Compare different simplification prompts or decoding temperatures.
- Apply word-sense disambiguation before calculating WordNet depth.
- Compare open-weight and proprietary LLMs.
- Extend the analysis to multilingual text simplification.
- Study whether model size influences lexical diversity and meaning preservation.

Project 21: Decentralized LLM-Agent Coordination under Robot Limitations (Reserved for Minttu Ukkola)

Goal

Design and evaluate a team of LLM-based robot agents that coordinate directly through natural-language communication in a constrained text-based environment. The project uses a **decentralized peer-to-peer architecture**. There is no central supervisor. Each robot has its own local observations, physical capabilities, memory, and received messages. The main objective is to investigate how realistic robot limitations, such as restricted capabilities, partial observability, limited communication, carrying capacity, and shared resources, affect task completion and communication behavior. A passive observer records the complete execution trace. Objective performance is calculated by the simulator, while an independent LLM judge evaluates semantic properties of communication such as clarity, relevance, grounding, and coordination utility.

Environment

No external dataset is required. The project will create a small benchmark of text-based multi-robot tasks in a deterministic kitchen environment.

Possible locations include Refrigerator, Cutting station, Sink, Stove, Assembly station, and Serving table

The environment contains structured information: Robot locations and capabilities, Objects, ingredients, and tools, Available and occupied workstations, Task dependencies, Agent messages, Completed and failed actions

The LLM proposes actions, but the Python simulator determines whether each action is valid.

For example, CUT(tomato) may require:

- The robot to be at the cutting station.
- The tomato to be at the cutting station.
- A knife to be available.
- The robot to have cutting capability.

This prevents an agent from claiming that it completed an impossible action.

Specifications

1. Deterministic Text-Based Robot Environment

- Implement the environment as a state-transition system in Python.
- Define actions such as: MOVE, PICK, CUT, WASH, COOK, ASSEMBLE, MESSAGE, WAIT
- Define the preconditions, duration, resource requirements, and effects of every action.
- Reject invalid actions and return an informative error message to the acting agent.
- Store the complete state after every timestep.

2. Decentralized Peer-to-Peer Agent Architecture

- Use three or four robot agents with separate context and memory.
- Broadcast the user's task to all agents at the beginning of the run.
- Do not provide any agent with supervisory authority.
- Agents must communicate directly to: Propose task allocations, Announce capabilities, Request objects or assistance, Report task progress, Resolve resource conflicts and Revise plans after failures
- Each agent receives only its own observations, capability profile, memory, and received messages.

The general agent cycle is:

Observe → reason → communicate or act → receive feedback → update plan

3. Robot Capability Profiles

Give the agents equal underlying LLM capabilities but different physical attributes. Possible limitations include: Allowed action types, Carrying capacity, Tool compatibility, Workstation accessibility, Sensor range, Movement cost and Ability to manipulate or heat objects

For example: A carrier robot can move several objects but cannot cut/cook, A manipulation robot can cut and assemble but carries only one object, Heat-resistant robot can operate the stove with limited manipulation capabilities.

The agents should discover and communicate these differences rather than receiving a global allocation plan.

4. Multi-Robot Task Benchmark

Create approximately six tasks with different coordination requirements.

Difficulty	Example task	Main challenge

Easy	Prepare a salad	Basic task allocation
Easy	Prepare a sandwich and drink	Parallel activities
Medium	Prepare hamburgers	Sequential dependencies
Medium	Prepare soup and salad	Shared cutting tools
Hard	Prepare two hamburgers and a salad	Resource competition
Hard	Prepare a complete meal	Planning, dependencies, and parallel execution

For every task, define: Required final state, Task dependency graph, Required objects and tools, Minimum required capabilities and Expected opportunities for parallel execution

5. Controlled Robot-Limitation Conditions

Use an unconstrained reference condition and compare it with isolated limitation conditions.

Condition	Description
Reference	Full visibility, broad capabilities, and generous communication budget
Capability-limited	Robots have heterogeneous action sets and carrying capacities
Observation-limited	Robots see only their current location and nearby objects
Communication-limited	Maximum number of messages or communication tokens
Combined	Capability, observation, resource, and communication limitations occur together

The student may replace one condition with a memory limitation, such as allowing an agent to see only the last k messages. The individual limitation conditions are important because they show which restriction causes performance degradation. The combined condition represents a more realistic and difficult robotic scenario.

6. Natural-Language Communication Analysis

Store every inter-agent message together with: Sender, Recipient, Timestep, Local environment state, Previous action, and Intended communication function

Analyze communication functions such as: Proposal, Commitment, Request, Status update, Warning, Conflict resolution, Plan revision and Failure explanation

Compare how the frequency and content of these messages change under different limitations. For example, partial observability may increase information requests, while communication limits may produce shorter but less complete messages.

7. Passive Observer and LLM Judge

Implement a passive observer that receives: User instruction, Global environment states, Agent observations, Actions and action results, Agent messages, Resource conflicts, Constraint violations and Final task state

The observer must not communicate with the agents or influence execution.

Use deterministic simulator logic for factual measures such as success, invalid actions, and resource conflicts. Use an independent LLM judge only for semantic communication qualities: Relevance, Clarity, Grounding, Coordination utility, Redundancy, Planning coherence, Quality of failure explanations

The judge should return structured scores on a 1–5 scale with a short justification. If possible, use a different model from the acting agents.

Manually evaluate a small sample of messages or traces to check the reliability of the LLM judge. Report agreement or correlation between human and judge assessments.

8. Failure and Recovery Scenarios

Introduce a small number of controlled failures, particularly in the medium and hard tasks.

Examples include: A tool becomes temporarily unavailable, A workstation is occupied, An attempted action fails. An object is delivered to the wrong location., A status message is delayed or omitted.

Measure whether agents: Detect the failure, Communicate its calocation.op an alternative plan and Successfully complete the task afterward

Failures should be predetermined or seed-controlled so that conditions remain comparable.

9. Task-Performance and Coordination Metrics

Report at least the following objective metrics: Task success rate, Valid action rate, Constraint violation rate, Makespan or total completion time, Total actions, Total movements, Idle timesteps, Resource conflicts, Dependency errors, Replanning frequency, Recovery rate and Workload distribution across agents

Task success must be determined from the final simulator state rather than from an agent's claim.

10. Communication Metrics

Analyze: Number of messages, Total communication tokens, Messages per successful task, Tokens per successful task, Average message length, Grounded message rate, Redundant message rate, Communication-function

distribution, LLM-judge relevance and clarity scores, Coordination utility and Relationship between communication volume and task success

A message is ungrounded when it contradicts the environment state, for example, when an agent reports that an ingredient has been cut although the simulator records it as uncut.

11. Experimental Evaluation

Use the same: LLM model, Agent prompts, Robot count, Decoding parameters, Task definitions, Initial environment states and across conditions.

A manageable core experiment is: 6 tasks, 5 constraint conditions, 3 repetitions

This produces **90 executions**. Additional repetitions can be performed if API cost and time permit.

Compare every limitation condition against the reference condition. Report: Mean and standard deviation, Confidence intervals, Paired statistical comparisons, Effect sizes, Failure categories, Processing time, API latency and Token usage and estimated cost. Also examine whether communication characteristics such as grounding, relevance, and redundancy predict task success or recovery.

12. Visualization, Documentation, and Limitations

Develop a small Streamlit or Gradio interface that displays:

- Current robot and object locations
- Robot capability profiles
- Current task graph
- Inter-agent messages
- Executed and rejected actions
- Resource conflicts
- Final performance metrics
- LLM-judge communication scores

The interface may replay stored traces instead of making live API calls.

Provide: Source code, Environment configurations, Task definitions, Capability profiles, and Prompts

Discuss limitations such as:

- Difference between text simulation and physical robots
- Simplified perception and movement
- Dependence on the selected LLM
- Prompt sensitivity
- Stochastic model behavior
- Reliability and bias of the LLM judge
- Limited number of tasks and repetitions

Extensions (Optional)

- Compare two LLM families or model sizes.
- Add message loss or communication delay.
- Introduce battery or energy limitations.
- Examine longer-term memory limitations.
- Add more heterogeneous robot embodiments.

Project 22: FoodJudgeBench: Evaluating LLM Judges of Food Recommendations and Explanations Using Historical and Synthetic User Profiles (Reserved Arshman Tariq and his group)

This research develops a benchmark for comparing LLM judges with human evaluators when assessing personalized food recommendations and their explanations. It combines **profiles reconstructed from HUMMUS interaction histories** with **fully synthetic profiles**, and evaluates outputs produced by different recommendation and explanation-generation techniques. Humans serve as annotators; there is no live recommendation experiment with the original users.

HUMMUS contains real historical user interactions, alongside recipe, user, and review information. What we lack is direct access to those users to verify their complete preferences or collect their reactions to newly generated explanations. Therefore, the first part uses *historically grounded, inferred profiles*, while the second uses *explicitly constructed synthetic profiles*. HUMMUS is a relevant published resource, but its interactions are not ground-truth labels for explanation quality. HUMMUS paper, dataset repository

Step 1 - Define the research objectives

The primary objective is to determine whether LLM judges can reliably assess recommendation suitability and explanation quality for a given profile. A second objective is to investigate whether their reliability changes with the profile source and explanation-generation technique.

The research questions are:

- **RQ1:** How closely do LLM judgments agree with human assessments?
- **RQ2:** Does agreement differ between profiles reconstructed from HUMMUS and synthetic profiles?
- **RQ3:** Do LLM judges reliably compare explanations generated through different techniques?
- **RQ4:** Which judging best detect unsupported personalization, factual, and ignored constraints?

The project evaluates assessment reliability and perceived nudge appropriateness. Actual persuasion or behavior change is outside its scope.

Step 2 - Audit and prepare HUMMUS

Inspect the selected release to identify available recipe attributes, interactions, ratings, reviews, timestamps, and nutrition information. Document missingness, duplication, rating distributions, and differences in history length.

Create a cleaned recipe catalog with consistent units and comparison bases. Use preparation time, equipment, or other attributes only where available and sufficiently reliable. Preserve missing values instead of filling them with unverified LLM estimates. For historical evaluation, construct profile histories using interactions preceding a cutoff, where timestamps support this. Reserve later interactions for a separate recommendation test. Exclude held-out ratings and review text from profile construction, explanation generation, and judge.

Step 3 - Construct the two profile collections

Use a common schema, while preserving the different evidence available in each collection.

Component	HUMMUS-derived profiles	Synthetic profiles
Preference source	Earlier ratings, reviews, and recipe attributes	Explicitly specified preferences
Constraints	Included only when supported by evidence	Deliberately varied across scenarios
Health goals	Unknown unless explicitly supported	Explicit scenario goals
Evidence	Links to historical observations	Links to scenario specifications
Uncertainty	Recorded for inferred attributes	Explicit unknown or unspecified fields

For HUMMUS, combine statistical summaries with optional LLM extraction. For example, repeated positive ratings of recipes containing legumes may support a tentative preference. It does not establish health goal.

For synthetic profiles, systematically vary tastes, goals, cooking time, and practical constraints. Include competing priorities and incomplete information. Sample structured attributes, optionally generate a readable description.

Use HUMMUS recipes for both collections. This keeps the item domain consistent. Human reviewers check profile clarity and evidence support before downstream annotation. If adding hypothetical goals to a historical profile, label that case separately as an augmented scenario.

Step 4 - Generate a diverse recommendation pool

Use a small set of recommendation approaches, such as content-based matching, collaborative filtering where applicable, and health-aware reranking. Historical profiles can support collaborative methods through observed interactions. Fully synthetic profiles need methods that operate on explicit attributes; do not invent interaction histories simply to make every algorithm applicable.

Select a balanced pool of recommended recipes, alternatives, and some challenging mismatches. Record the recommendation method and its actual supporting signals.

Maintain two evaluation tasks:

1. **Recommendation assessment:** Compare recipes for a fixed profile.
2. **Explanation assessment:** Compare explanations for a fixed profile–recipe pair.

The second task isolates explanation quality from the quality of the underlying recommendation.

Step 5 - Generate explanations using multiple techniques

Include methods with different forms of evidence and generation, rather than only different LLM names.

Technique	How the explanation is produced
Template-based	Verified matching attributes are inserted into fixed sentences
Content-based	Shared ingredients, cuisines, or other attributes are described
Model-based	Actual recommendation contributions or model signals are verbalized
Direct LLM generation	An LLM receives the profile and recipe and writes an explanation
Evidence-grounded LLM generation	An LLM receives a structured set of permitted facts and supporting records
Contrastive explanation	The recommended recipe is compared with a specified alternative

Distinguish an explanation of **why the algorithm selected an item** from a **justification of why the item may suit the profile**. A plausible LLM justification is not necessarily faithful to the recommendation.

Treat nudging style as a separate experimental factor. For a selected subset, compare neutral wording with goal-framed wording while preserving the facts. This allows to separate generation-method effects from framing effects.

Step 6 - Build standardized evaluation cases

Each case should include a profile, supporting profile evidence, recipe attributes, explanation, and any comparison recipe or recommendation evidence needed to assess its claims.

Assign identifiers for profile source, recommender, explanation technique, generator, and framing condition. Hide method and model identities during annotation.

Create additional diagnostic cases with controlled errors: an invented preference, an incorrect nutritional comparison, an ignored time constraint, or unsupported social proof. Keep these separate from naturally generated outputs in reporting.

Do not automatically repair all generated explanations. Errors are part of what the benchmark should measure. Exclude only unusable outputs under documented rules.

Step 7 - Define the human evaluation rubric

Humans evaluate suitability for the supplied profile, not according to their own tastes.

Dimension	Human evaluation question
Preference fit	Does the recommendation match the available preference evidence?
Constraint satisfaction	Does it respect explicit requirements?
Goal alignment	Does it support stated goals, where known?
Factual grounding	Are the claims supported by the supplied evidence?
Personalization fidelity	Does the explanation avoid inventing user attributes?
Clarity and trade-offs	Does it clearly communicate relevant reasons and limitations?
Nudge appropriateness	Is the framing proportionate and respectful of the stated goals?

Use explicit violation labels for objective errors and anchored ordinal ratings for subjective dimensions. Include “not applicable” and “insufficient evidence.”

For historical profiles, annotators assess **consistency with available evidence**, not the unknown person’s true preference. Goal alignment should be unscored when no goal is documented.

Step 8 - Collect independent human annotations

Pilot the rubric on a small, balanced sample. Revise confusing criteria before collecting the main annotations.

Use trained annotators for evidence consistency, clarity, and practical suitability. Assign claims requiring nutritional interpretation to nutrition-qualified evaluators. Numerical comparisons can additionally receive deterministic checks.

Obtain multiple independent assessments for the core test cases, ideally three per case. Collect dimension ratings, flagged errors, and short evidence-based justifications. Include a balanced subset of pairwise comparisons between explanations for the same recipe.

Adjudicate objective disagreements while retaining the original labels. Preserve subjective disagreement as a distribution of judgments rather than treating every disagreement as annotation error.

Step 9 - Run LLM judges under matched conditions

Give LLM judges the same evidence and rubric available to humans. Compare generic rubric-based judging, explicit constraint-checking, and tool-assisted judging that verifies numerical claims.

Use several judge model families and record repeated-run variability. Reverse pairwise presentation order to measure position sensitivity. Test whether judges systematically favor explanations produced by their own model family.

Develop prompts using development profiles only. Freeze configurations before evaluating held-out profiles and hidden human annotations. Keep related synthetic profile variants together in one split.

Step 10 - Evaluate agreement and generalization

Compare LLM and human evaluations using error-detection precision and recall, ordinal agreement, pairwise preference agreement, and rankings of explanation techniques. Report performance separately for both profile sources and each evaluation dimension.

Test whether conclusions remain stable across sparse versus richer histories, uncertain profiles, unseen recipe groups, and unfamiliar explanation techniques. Use profile-level resampling for uncertainty estimates because multiple outputs from one profile are related.

Held-out HUMMUS interactions provide an additional test of recommendation performance, using metrics such as Recall@K or NDCG@K. They do not validate newly generated explanations, and unobserved recipes should not automatically be treated as disliked.

Step 11 - Release the benchmark and report the findings

Release permitted data or reconstruction scripts, profile evidence, generated outputs, human annotations, diagnostic cases, judge configurations, and evaluation code. Document dataset versions, redistribution permissions, splits, and annotation procedures.

A practical pilot could include **30 historical and 30 synthetic profiles, two recipes per profile, and four explanation techniques**, yielding 480 explanation cases. Expand after measuring annotation time and reliability; the final release need not exhaust every possible combination.

Project 23: Agentic RAG for Evidence-Based Question Answering

Goal:

Build a question-answering system that retrieves evidence, identifies missing information, performs additional searches when necessary, and generates answers with supporting citations. Compare a fixed retrieval pipeline with an agent that controls its retrieval process.

Dataset:

Use a subset of HotpotQA, which provides multi-hop questions, answers, and sentence-level supporting facts. For a manageable implementation, use its distractor setting rather than searching all of Wikipedia.

Suggested scope: 100 development questions and 300 evaluation questions; a maximum of three retrieval steps per question.

Research question:

When does adaptive retrieval improve answer correctness and evidence coverage, and is the improvement worth the additional computation?

This reflects recent work on evaluating the individual retrieval and reasoning steps of agentic RAG, rather than measuring only final-answer accuracy. ACL Anthology

Specifications:

1. **Dataset exploration and preparation:** Analyze question lengths, answer types, and the number of supporting documents. Inspect examples requiring comparison or combining information across documents. Keep gold answers and supporting-fact labels hidden from the answering system.
2. **Document representation and indexing:** Preserve document titles and sentence identifiers. Implement BM25 and dense embedding retrieval over the supplied candidate passages. Compare their ability to retrieve supporting evidence.
3. **Baseline question-answering systems:** Implement a direct-answer LLM baseline and a standard RAG baseline that retrieves once and then answers. Use the same generator model for both.
4. **Question decomposition:** Ask the model to generate up to three short subquestions. For example, a question comparing two people's birthplaces may require identifying each birthplace before making the comparison. Store the subquestions as structured data.
5. **Agentic retrieval loop:** Provide tools such as search(query) and read_passage(document_id). At each step, the agent must choose whether to search again, read a passage, answer, or abstain. Enforce a fixed step limit and log every action.
6. **Evidence-based answer generation:** Return an answer, a short justification, and supporting sentence identifiers. Check that cited identifiers exist; separately assess whether the cited text actually supports the answer.
7. **Missing-evidence experiment:** Remove one annotated supporting passage from selected examples. Measure changes in correctness, unsupported claims, and abstention. Manually check whether remaining passages contain alternative support before treating a question as unanswerable.
8. **Evaluation and error analysis:** Report answer exact match and token F1, supporting-fact precision/recall, tool calls, token usage, and latency. Categorize failures as retrieval errors, reasoning errors, unsupported answers, or unnecessary searches. Include a fixed multi-query retrieval baseline to help distinguish adaptive planning from simply performing more searches.

Deliverables: Retrieval index, baseline and agent implementations, action logs, evaluation notebook, and comparison report.

Optional extensions: Hybrid retrieval, reranking, or a Streamlit interface showing the retrieved evidence and search sequence.

Project 24: Multi-Agent Debate for Reliable Reasoning

Goal:

Implement a framework in which multiple LLM agents independently answer questions, inspect each other's arguments, and revise their answers. Investigate whether debate improves reasoning or causes agents to abandon correct answers.

Dataset:

Use ARC-Challenge, a multiple-choice science-question dataset with official training, validation, and test splits. Preserve the original answer options.

Suggested scope: 100 development questions, 300 test questions, three agents, and at most two discussion rounds.

Research question: *Does agent interaction improve reasoning beyond independent voting, and when does it introduce harmful agreement?*

Independent voting is an essential baseline: recent research finds that debate does not consistently outperform simpler aggregation methods. arxiv.org

Specifications:

1. **Dataset exploration:** Analyze question length, number of options, and common scientific concepts. Manually categorize a small sample into factual, causal, comparison, and numerical reasoning questions.
2. **Single-agent baselines:** Compare direct answering with answering accompanied by a short justification. Store the predicted option, justification, and generation metadata.
3. **Independent multi-agent answering:** Run three independent model calls without sharing responses. Aggregate answers using majority voting and define a reproducible tie-breaking rule. Initially use the same model to isolate the effect of interaction.
4. **Debate protocol:** Give each agent the other agents' previous answers and arguments. Ask it to identify a specific strength or weakness before revising. Use synchronous rounds so all agents see the same previous-round information.
5. **Structured criticism:** Compare unrestricted discussion with a structured response containing: proposed answer, supporting argument, criticism of an alternative, and final revised answer. Keep response-length limits comparable.
6. **Final decision methods:** Compare majority voting after discussion with an LLM judge that receives the anonymized final arguments. The judge must not receive the gold answer. Record whether the decision method changes the outcome.
7. **Answer-change analysis:** Track correct-to-incorrect and incorrect-to-correct revisions. Measure how often a correct minority persuades the group and how often an incorrect majority pulls a correct agent toward its answer. Report these rates with their relevant denominators.
8. **Quality–cost comparison:** Report accuracy, disagreement, answer-change rates, tokens, and latency. Include single-agent self-revision or additional independent samples under a similar total token budget. Manually inspect examples of useful criticism, unsupported persuasion, and repeated arguments.

Deliverables: Debate runner, prompt templates, complete transcripts, round-by-round analysis, and comparative results.

Optional extensions: Different model families, agents with different evidence subsets, or early stopping when further discussion is unlikely to help.

Important experimental principle: Better performance is a hypothesis to test; a well-supported finding that debate hurts performance is also a successful project outcome.

Project 25: Conversational Recommendation Agent with Memory and Tool Use

Goal:

Build a conversational recipe assistant that extracts user requirements, remembers preferences across turns, calls retrieval and validation tools, and explains its recommendations using dataset evidence.

Dataset:

Use the **HUMMUS recipe data** after checking the actual supplied fields and nutritional units. Select approximately 5,000 recipes with sufficient ingredient, preparation-time, and nutritional information.

Suggested scope: 100 dialogue scenarios, each containing three to five user turns; 30 development scenarios and 70 held-out evaluation scenarios.

Research question: *Do structured memory and explicit tool use improve constraint satisfaction as user requirements change during a conversation?*

Recent agent benchmarks explicitly investigate clarification, user interaction, and handling infeasible instructions, making these useful capabilities to study in a small, controlled environment. proceedings.mlr.press

Specifications:

1. **Recipe exploration and normalization:** Inspect missing values, ingredient aliases, dietary tags, preparation times, and nutritional units. Establish whether values are per serving or per 100 g before implementing numerical filters.
2. **Dialogue dataset construction:** Create scenarios covering initial requests, added constraints, changed preferences, and infeasible combinations. For example: "Suggest a vegetarian dinner," followed by "Under 25 minutes," and then "Avoid mushrooms." Store the expected active constraints after every turn.
3. **Natural-language constraint extraction:** Convert requests into a structured schema containing included/excluded ingredients, dietary preferences, maximum duration, and numerical limits. Compare a simple rule-based parser with LLM-based extraction.
4. **Dialogue state and memory:** Maintain persistent preferences separately from temporary requests. Distinguish additions from replacements: "Actually, up to 30 minutes is fine" should update the earlier time limit. Record where each constraint originated.
5. **Retrieval and validation tools:** Implement tools such as search_recipes, get_recipe_details, and validate_constraints. Use deterministic code to check numerical and ingredient constraints rather than relying on the LLM's judgment.
6. **Agent decision policy:** Let the agent choose whether to retrieve, inspect a candidate, ask a clarification question, or explain that no matching recipe was found. It must not silently relax an explicit constraint. Use scripted user responses to make evaluation reproducible.
7. **Grounded recommendation explanations:** Generate explanations referring to verified recipe attributes and active user requirements. If the user requests competing objectives, show the trade-off explicitly, for example, the quickest option versus the higher-protein option.
8. **Evaluation and ablation study:** Compare a standard conversational RAG baseline, RAG with structured memory, and the full tool-using agent. Measure constraint-extraction F1, dialogue-state accuracy, percentage of recommendations satisfying all active constraints, unsupported factual claims, appropriate handling of infeasible requests, and tool usage. Disable memory or validation separately to assess their contributions.

Deliverables: Recipe database, annotated dialogue scenarios, tool implementations, conversation logs, and evaluation report.

Optional extensions: Preference-based ranking, longer conversations, or a reviewer agent that checks explanations against tool outputs.

Evaluation boundary: These scenarios test simulated task completion. They do not establish real-user satisfaction or long-term behavioral effects.

Project 26: Do LLM Explanations Track the Evidence? Counterfactual Evaluation of Prediction–Explanation Consistency (Bigger Project with the possibility of a research publication)

Goal: Develop an experimental framework to investigate whether LLM predictions and explanations respond appropriately to changes in input evidence. Students will compare explanation-generation strategies, construct controlled input modifications, and analyze cases where a model produces a correct answer with an incorrect explanation, or a convincing explanation for an incorrect answer. The project focuses on **the behavior of the model producing predictions and explanations**. Human review will validate experimental examples and selected outputs; automated judging will be an optional supporting component.

Dataset

Use **e-SNLI** (**https://github.com/OanaMariaCamburu/e-SNLI**), which provides premise–hypothesis pairs, natural-language inference labels, human-written explanations, and highlighted evidence.

The prediction task has three labels:

- **Entailment:** The hypothesis follows from the premise.
- **Contradiction:** The hypothesis conflicts with the premise.
- **Neutral:** The premise provides insufficient information to establish or contradict the hypothesis.

Human explanations and highlights serve as reference evidence. They are not treated as the only possible valid explanations.

Suggested scope

Use approximately **100 development examples and 300 held-out evaluation examples**, balanced across the three labels. Select **100 evaluation examples** for controlled modifications, producing three additional versions per selected example. This gives approximately **600 evaluation inputs**: 300 original examples and 300 modified examples. Keep every original example and its variants in the same split.

Research questions

- **RQ1:** How do explanation-generation strategies affect prediction accuracy and explanation correctness?
- **RQ2:** When decisive evidence changes, do the prediction and explanation change appropriately?
- **RQ3:** Are predictions and explanations stable under meaning-preserving changes and irrelevant information?
- **RQ4:** Do explanations identify evidence whose modification affects the model's decision?
- **RQ5:** Can a verification-and-revision step improve explanations without degrading correct predictions?

Specifications

1. Dataset exploration and preparation

Inspect label balance, premise and hypothesis lengths, explanation lengths, and annotated evidence.

Manually categorize a sample into reasoning phenomena such as negation, quantities, actions, entity attributes, and missing information. These categories will support later error analysis.

Preserve negations, quantities, punctuation, and sentence structure. Avoid preprocessing that could change the inference relation.

Store each example with its identifier, input sentences, gold label, reference explanation, and available evidence annotations. Gold labels and reference explanations must not be included in evaluation prompts.

2. Implement prediction and explanation baselines

Use the same underlying LLM to compare four conditions:

Condition	Required output
Label only	Predicted inference label
Label first	Label followed by a short explanation
Evidence first	Relevant quoted spans, a short explanation, then the label
Structured explanation	Label, supporting spans, justification, and any information that is missing

Use comparable output limits across explanation conditions. Freeze prompts after development.

Compare at least two model configurations if resources allow. Record model versions, prompts, decoding settings, token usage, and latency.

3. Generate concise, evidence-linked explanations

Require the model to identify relevant spans from the premise and hypothesis and explain their relationship.

Premise: "No participants completed the task."
Hypothesis: "Some participants completed the task."
Prediction: Contradiction.
Evidence: "No participants completed" and "Some participants completed."
Explanation: The premise rules out any participant completing the task, whereas the hypothesis asserts that at least one did. Validate that quoted spans occur in the input. Evaluate their relevance separately: an existing quotation is not necessarily adequate support.

For neutral examples, allow the explanation to identify missing information rather than forcing it to quote evidence that does not exist.

4. Construct three types of controlled input modifications

For each of the 100 selected examples, create one version of each type:

Modification	Purpose	Expected behavior
Label-changing edit	Change decisive evidence while keeping other content as similar as possible	Update the prediction and the relevant explanation
Meaning-preserving paraphrase	Change wording while preserving the inference relation	Preserve the label and the explanation's substantive meaning
Irrelevant-information addition	Add information that does not affect the inference relation	Preserve the label and avoid treating the addition as decisive evidence

Example:

Version	Premise	Hypothesis	Gold label
Original	"No participants completed the task."	"Some participants completed the task."	Contradiction
Label-changing edit	"Some participants completed the task."	Same	Entailment
Paraphrase	"Not a single participant finished the task."	Same	Contradiction
Irrelevant addition	"No participants completed the task. The room had blue walls."	Same	Contradiction

An LLM may propose modifications, but reviewers must verify their labels, naturalness, and intended effects. Store the changed span and the reason for each expected label.

5. Measure prediction–explanation consistency across versions

Evaluate each original and modified input independently, without revealing the paired version to the model.
Then analyze whether the model:

- Produces the correct label before and after modification.
- Updates its explanation when decisive evidence changes.
- Preserves the substantive explanation under paraphrasing.
- Introduces unsupported reasoning after irrelevant information is added.
- Changes its answer but repeats an explanation that supports the previous answer.

Measure both **paired correctness** and the appropriateness of the explanation change. An answer flip alone is not success: both predictions may be wrong.

6. Test whether model-cited evidence matters to its decision

On a smaller subset, inspect the evidence cited in the model's original explanation.
Construct two controlled interventions where feasible: Modify a cited, relevant span., Modify an uncited span of comparable size.
Human reviewers should independently classify whether each edit changes the correct inference relation. Avoid simple word deletion when it produces unnatural sentences or unclear labels. Examine whether model behavior tracks **semantic relevance**, **its own cited evidence**, or neither. Report cases where decisive evidence was omitted from the explanation. These tests provide behavioral evidence about explanation faithfulness; they do not reveal the model's internal reasoning process.

7. Implement a verification-and-revision method

Build a lightweight second pass that receives the input, predicted label, and explanation. Ask it to check: Do the quoted spans exist? Does the explanation support the predicted label? Does it introduce facts absent from the input? Does it confuse missing information with contradiction?
The system then returns a revised prediction and explanation, or retains the original.
Compare this method with a generic "check your answer" self-revision baseline under a similar inference budget. The verifier's approval must not be used as the evaluation ground truth.

8. Evaluate prediction quality, explanation quality, and robustness

Use complementary metrics:

Dimension	Suggested measurement
Prediction quality	Accuracy and macro-F1
Counterfactual performance	Percentage of original–edited pairs with both labels correct

Paraphrase robustness	Label consistency, alongside accuracy on both versions
Irrelevant-information robustness	Prediction changes and accuracy degradation
Evidence validity	Percentage of quoted spans actually present in the input
Explanation correctness	Human-rated support for the inference relation
Prediction–explanation consistency	Whether the explanation supports the returned label
Explanation adaptation	Whether the explanation correctly addresses the changed evidence
Unsupported reasoning	Percentage of explanations introducing unsupported claims
Revision effectiveness	Incorrect-to-correct changes and correct-to-incorrect changes
Efficiency	Tokens, latency, and additional calls

Do not rely on BLEU or ROUGE as the main explanation metrics: explanations can express the same valid reasoning in different words.

9. Conduct a focused human evaluation and error analysis

Select approximately **100 original–modified output pairs**, balanced across modification types, labels, and generation strategies.

Two reviewers independently assess the outputs using a short rubric. Hide model and method identities. Report agreement before adjudication.

Classify recurring failures, including:

- Correct prediction with an invalid explanation.
- Incorrect prediction with a fluent justification.
- Prediction updated but explanation left inconsistent.
- Unsupported assumptions that convert neutral cases into entailment or contradiction.
- Sensitivity to irrelevant details.
- Failure to recognize a meaningful evidence change.

Human evaluation should assess reasoning against the supplied sentences, rather than reviewers' preferences for writing style.

10. Compare methods and report reproducible findings

Compare the explanation strategies and revision methods on the same examples. Use paired analyses and confidence intervals clustered by original example, because modified versions are related observations.

Report results separately by label, modification type, and reasoning phenomenon. Include an ablation removing explicit evidence quotation to test whether quoting evidence contributes beyond structured formatting.

Deliver:

- Cleaned evaluation data and validated modifications.
- Prompt templates and experiment code.
- Predictions, explanations, and evidence spans.
- Human annotation rubric and labels.
- Evaluation tables, robustness plots, and a failure taxonomy.
- A report discussing what the results establish—and the limits of behavioral faithfulness tests.

Optional extensions

- Add misleading external hints, such as another system's incorrect suggested label.
- Test whether longer explanations improve evidence sensitivity or merely increase verbosity.
- Examine transfer to a second inference dataset.
- Use an automated evaluator for preliminary screening, validating its decisions against human annotations.

Expected outcome

The project should establish **when explanations remain aligned with changing evidence, which prompting strategies improve that alignment, and whether verification repairs errors or introduces new ones**.

Project 27: Multi-Agent Judges for Conflicting Recommendation Objectives (Bigger Project with the possibility of a research publication)

Goal: Develop and evaluate an LLM-based judging framework for recommendations involving competing objectives. Investigate whether specialized judges improve evaluation reliability, and whether persuasive explanations can make judges approve inferior recommendations.

The system evaluates **recommendation outputs**, rather than primarily trying to build a better recommender.

Why this could support a publication

Multi-objective selection through natural-language preferences already exists—for example, LISTEN studies this problem. Therefore, "using an LLM to balance objectives" is insufficient as a new contribution. IJCAI

Likewise, judge consistency and multi-judge calibration are active research topics. arxiv.org

The proposed contribution is more specific:

A controlled evaluation of whether judges respect verifiable constraints, recognize genuine trade-offs, and resist explanation-induced changes when the underlying recommendation stays identical.

Research questions

- **RQ1:** How reliably do LLM judges distinguish constraint violations, dominated recommendations, and legitimate trade-offs?
- **RQ2:** Does separating evaluation into specialist judgments improve reliability compared with a single judge given the same rubric?
- **RQ3:** Can fluent or persuasive explanations change judgments without changing recommendation quality?
- **RQ4:** Can selective discussion preserve evaluation quality while reducing the cost of a multi-agent panel?

Dataset and benchmark

Use the HUMMUS recipe data. Begin with a verified subset of approximately **5,000** recipes.

Construct approximately **600 independent evaluation cases**. Each case contains:

- A user request and a clearly specified preference profile.
- A fixed candidate pool with verified recipe attributes.
- Two competing recommendation outputs.
- Their explanations.
- Ground-truth labels for objectively checkable properties.

Organize cases into three groups:

Case type	Example	Ground truth
Constraint violation	A recommended recipe takes 45 minutes despite a strict 20-minute limit	Deterministic validation
Dominated recommendation	One option is no better on any declared objective and worse on at least one	Computed from the specified objective vector
Genuine trade-off	One option better matches preferences; another has a better nutritional score	No universal winner without an explicit preference policy

This distinction is essential. Human preference, a benchmark utility function, and factual correctness should not be treated as interchangeable labels.

Specifications

1. **Define the evaluation policy:**

Separate hard constraints from soft objectives. For example, an explicitly excluded ingredient is a hard constraint; preparation speed may be a soft preference unless the user specifies a strict limit.

Define the meaning and direction of each objective. Treat a nutritional score as a dataset-based proxy, not a measured long-term health outcome.

2. **Construct user profiles**

Create profiles with preferred ingredients, disliked ingredients, time limits, and explicit priorities.

Use controlled synthetic profiles for the core benchmark. If interaction histories are available, add a separate realism experiment using profiles derived only from historical data. Do not describe a similarity score as ground-truth user satisfaction.

3. **Generate recommendation pairs**

Produce correct, constraint-violating, dominated, and trade-off pairs programmatically. Include easy and subtle violations.

Verify that neither candidate quality nor correctness can be inferred from ordering, formatting, recipe identifiers, or explanation length.

4. **Build explanation variants**

For the same recommendation, create: A concise factual explanation., A longer but factually equivalent explanation., A persuasive explanation that omits an important disadvantage., An explanation containing a specific false claim.

Manually check the variants. Analyze harmless style changes separately from changes in factual content.

5. **Implement strong baselines**

Compare a deterministic validator, a single holistic LLM judge, a single judge using a structured rubric, and independent specialist judges with fixed aggregation.

The structured single-judge baseline should receive the **same evidence and evaluation criteria** as the proposed system.

6. **Develop the proposed multi-agent evaluator**

Use three specialist roles:

- **Preference judge:** evaluates consistency with the stated user profile.
- **Attribute judge:** checks numerical facts and hard constraints using tools.
- **Explanation judge:** checks factual support and disclosure of relevant trade-offs.

Each returns a structured assessment with evidence references. A coordinator combines the assessments using an explicit policy. Verified hard-constraint failures cannot be overturned by a majority vote.

7. **Add selective discussion**

Trigger one discussion round only when judges disagree on a shared factual claim or when the aggregation policy cannot resolve the case.

Differences between objectives are not automatically "disagreement": one recipe can correctly score better for preference and worse for preparation time.

8. **Evaluate judge reliability**

Report:

- Precision and recall for detecting constraint violations.
- Accuracy on dominated-pair comparisons.
- Alignment with explicitly declared priorities.
- Decision stability under candidate-order swaps.
- Decision changes under factually equivalent explanation variants.
- False-claim detection.
- Token usage, latency, and discussion frequency.

Allow "insufficient preference information" for genuinely unresolved trade-offs, and report its frequency separately.

9. **Conduct human validation**

Select approximately **100–200 cases** for independent annotation by two reviewers (preferably outside of group member). Annotators should rate factual validity separately from subjective preference.

Report agreement and adjudicated labels. Preserve disagreements on legitimate trade-offs rather than forcing every case into one "correct" ranking.

10. **Run controlled ablations and statistical analysis**

Remove tools, specialist roles, explanation checking, and selective discussion one at a time. Compare at similar inference budgets. Keep all variants of an original case in the same data split. Use paired comparisons and bootstrap confidence intervals clustered by original case or profile; explanation variants are not independent samples.

Expected publication package

- A documented benchmark with objective labels and explanation perturbations.
- A reproducible judging framework.
- Cross-model evaluation using at least three model families or configurations.
- A taxonomy of evaluation failures.
- Evidence showing when specialist judging helps and when a simpler evaluator is sufficient.

What would make the findings publishable?

A useful result could be that judges reliably detect explicit violations but fail under persuasive explanations—or that tool-backed judging helps substantially while debate adds little. The contribution should be the demonstrated mechanism and reproducible evidence, not merely a higher average score.

Optional extension: Add a disclosed provider objective, such as a synthetic promotion score, to investigate whether judges confuse commercial benefit with user benefit. Keep this outside the initial core experiment.

Project 28: When Should an AI Agent Remember, Update, or Ignore a Preference?

(Bigger Project with the possibility of a research publication)

Goal: Build an agent that maintains user preferences across multiple conversations while distinguishing permanent changes, temporary exceptions, hypothetical statements, and requests made on behalf of another person. Evaluate both **what the agent remembers** and **how that memory changes its recommendations/ explanations**.

LongMemEval already evaluates long-term memory abilities, including temporal reasoning and knowledge updates. More recent work directly investigates the difficulty of maintaining and updating bounded agent memory. Therefore, "adding memory" or "handling contradictions" alone is not a sufficient contribution. Paper: web.cs.ucla.edu and Code: https://github.com/xiaowu0162/LongMemEval

The proposed contribution is narrower:

A benchmark of context-dependent preference changes, with matched conversation variants that trace errors from memory updates to downstream decisions and explanations.

The important distinction is between a new preference and a statement that applies only in a particular context.

Example

A user says:

1. "I usually prefer vegetarian meals."
2. "For tonight, suggest a chicken dish for my brother."
3. "Tomorrow, recommend lunch for me."

A reliable agent should understand that the second request concerns another person and a specific occasion. It should not permanently replace the user's own preference.

Research questions

- **RQ1:** Which types of conversational updates cause agents to overwrite, retain, or apply preferences incorrectly?
- **RQ2:** Does storing scope and provenance improve recommendations beyond chronological memory or conversation summaries?
- **RQ3:** Can an agent recall a preference correctly but still fail to apply it during tool use?
- **RQ4:** When do explanations incorrectly attribute a recommendation to something the user supposedly said?

Dataset and benchmark

Use a fixed recipe catalogue from HUMMUS to make recommendation validity programmatically checkable.

Construct approximately **240 base conversation histories**, each spanning **6–10 sessions** with explicit timestamps. Add one matched variant for each history, producing roughly **480 trajectories**.

The matched variant changes one crucial detail while preserving the rest of the conversation.

Update type	Example	Expected handling
Permanent revision	"I now prefer mild food."	Update the ongoing preference
Temporary exception	"Only for tonight, spicy is fine."	Apply within the stated period
Request for someone else	"My brother likes spicy food."	Associate with the brother
Hypothetical statement	"If I liked spicy food, what would you suggest?"	Do not update the actual profile
Explicit correction	"You misunderstood; I said no mushrooms."	Correct the relevant record
Ambiguous update	"That restriction is no longer important."	Resolve the referent or ask

Gold annotations specify the active profile, relevant historical evidence, and acceptable actions at selected checkpoints.

Specifications

1. **Define preference-update semantics**

Write annotation rules for permanent updates, exceptions, corrections, expiry, and requests for other people. Distinguish the time when a statement was made from the period when it applies. Establish when clarification is required.

2. **Build a structured scenario generator**

Generate an underlying event sequence first, including the gold state after each event. Then convert events into natural dialogue through templates, human writing, and controlled paraphrasing.

This avoids asking an LLM to invent both the conversation and its supposedly authoritative answer key.

3. **Create a human-authored challenge set**

Include at least **50 independently written histories** using indirect references, natural corrections, and ambiguous language. Keep these separate from template-based development data.
Have two reviewers validate a stratified sample of the benchmark and all challenge-set labels.

4. **Implement memory baselines**

Compare: Recent conversation only., Full conversation history, where it fits., A rolling natural-language summary., Retrieval over past utterances., A structured profile using the latest statement for each field.
Include a **gold-state baseline** that receives the correct active profile, providing an estimate of performance when memory errors are removed.

5. **Develop the proposed memory method**

Store each preference with fields such as:
subject, attribute, value, scope, valid_from, valid_until, source_turn, and supersedes.
Use the LLM to extract candidate updates. Apply explicit rules to resolve unambiguous temporal and contextual relationships; request clarification when required.
Retain superseded records for historical questions while excluding them from current decisions.

6. **Integrate a fixed recommendation pipeline**

Connect each memory method to the same recipe search and constraint-validation tools. Keep retrieval, ranking, and generation fixed across experiments so differences can be attributed to memory handling.
At each checkpoint, require an active-profile output, a recommendation or clarification action, and a short explanation.

7. **Separate memory errors from action errors**

Run two evaluations:

- **State evaluation:** Did the agent reconstruct the correct active preferences?
- **Action evaluation:** Did its recommendation satisfy those preferences?

Repeat selected cases with the gold profile supplied. This reveals whether the failure occurred during memory processing or downstream reasoning.

8. **Audit explanation provenance**

Require explanations such as “I excluded mushrooms because you asked me to” to cite the relevant conversation turn.
Check whether the cited turn exists, supports the claim, concerns the correct person, and remains applicable. Distinguish faithful references from invented user preferences.

9. **Evaluate robustness and recovery**

Vary irrelevant conversation length, distance since the update, number of revisions, and linguistic explicitness.
Measure: Active-preference precision, recall, and exact-state accuracy., Rate of applying another person’s preference to the user., Rate of applying expired or superseded preferences., Recommendation constraint satisfaction., Appropriate clarification and unnecessary clarification., Explanation provenance accuracy., Recovery after correction., Memory size, tokens, and latency.
Report paired performance on matched histories: does changing “for tonight” to “from now on” produce the appropriate behavioral change?

10. **Run ablations and generalization tests**

Remove scope, timestamps, provenance, and update resolution separately. Split data by scenario family and profile, keeping matched histories together.
Evaluate the best methods on the human-authored challenge set and a relevant subset of an existing memory benchmark. Use trajectory-level confidence intervals because checkpoints within one history are correlated.

Expected publication package

- A longitudinal benchmark with executable state and action labels.
- A method for applying preferences within their correct context.
- A diagnostic evaluation connecting memory, actions, and explanations.
- Results across multiple model configurations.
- Released scenario-generation code, evaluation scripts, and failure examples.

What would make the findings publishable?

A strong finding would show that an agent can answer “What does the user prefer?” correctly while still recommending an incompatible item, or that a temporary request persistently contaminates later personalization. Demonstrating these failure modes, and identifying which memory components prevent them, is more informative than reporting only aggregate recall accuracy.

Optional extension: Introduce a second domain to test transfer. Keep the core study focused on one domain until the evaluation is stable. Simulated longitudinal trajectories support claims about system behavior, not actual changes in users’ preferences or well-being.

Project 29: Contextual Preference Extraction from Food Reviews

Goal: Develop an NLP-based food recommender that identifies **what users like, under which circumstances, and whose preferences they describe**, using NER, topic modeling, aspect-based sentiment analysis, and relation extraction.

For example:

"I love spicy food, but this was too hot for my children. Great for weekend dinners, but too time-consuming for weekdays."

A simple sentiment model may classify this review as mixed. The proposed system should extract separate contextual preferences:

- Reviewer → likes → spicy food.
- Children → find → this dish too spicy.
- Recipe → suitable for → weekend dinners.
- Recipe → unsuitable for → time-constrained weekdays.

Dataset: HUMMUS DATASET.

Topic modeling and context-aware recommendation already have established precedents. The Project is the joint extraction of **preference holder, aspect, sentiment, and contextual condition**, and testing whether this prevents incorrect generalization in user profiles. Its novelty should be established through a fuller literature review. arxiv.org

Specifications:

1. **Exploratory Data Analysis and Text Preparation**

Analyze review lengths, missing values, rating distributions, and the frequency of contextual expressions such as "weeknight," "for my children," and "special occasion." Split reviews into sentences while preserving negation and contrast words such as "not," "but," and "although."

2. **Named Entity Recognition for Food and Context**

Extract ingredients, dishes, cooking equipment, durations, occasions, and groups of diners. Compare dictionary-based matching with a domain-adapted NER model. General-purpose NER requires additional labels or training for entities such as OCCASION, DINER_GROUP, and COOKING_EQUIPMENT.

3. **Topic Modeling of Cooking Experiences**

Compare LDA, NMF, and BERTopic to discover themes such as quick meals, family cooking, meal preparation, celebrations, and cooking difficulty. Inspect representative reviews and assess topic coherence, diversity, and human interpretability. Treat discovered topics as exploratory themes rather than verified user attributes.

4. **Aspect-Based Sentiment Analysis**

Extract sentiment toward taste, texture, preparation effort, time, ingredient availability, and perceived healthiness. Allow different sentiments within the same review. For example, "delicious but difficult" should produce positive taste sentiment and negative preparation-effort sentiment. Keep perceived healthiness separate from nutritional measurements.

5. **Preference-Holder and Relation Extraction**

Identify whether an opinion belongs to the reviewer, their children, a partner, or guests. Extract structured tuples:
\[(\text{holder},\text{aspect},\text{sentiment},\text{context},\text{evidence span}) \]
Compare dependency-parsing rules with a supervised relation extractor or an LLM using a fixed output schema. Assign "unknown" when the holder or context is ambiguous.

6. **Negation and Conditional Preference Detection**

Distinguish statements such as "I dislike spicy food," "not too spicy," and "I enjoy spicy food, except at breakfast." Detect negation scope, exceptions, and contrasts. Construct a small manually checked challenge set of minimally different sentences to test whether these linguistic changes produce the correct extracted preferences.

7. **Contextual User Profile Construction**

Build separate profiles for general preferences and explicitly mentioned contexts, such as weekday cooking or meals for children. Retain evidence spans and extraction confidence. Do not transfer a child's dislike into the reviewer's general preferences or interpret one situational comment as a permanent preference.

8. **Context-Aware Recipe Retrieval and Recommendation**

Accept requests such as:

"Recommend a quick dinner for my children."
Combine recipe-content similarity, collaborative filtering, and compatibility with the relevant contextual profile. Compare against popularity, CF, review embeddings, and a context-independent profile using the same extracted facts. Use structured recipe attributes for checkable constraints such as preparation time.

9. **Evidence-Grounded Summarization and Explanations**

Generate short contextual summaries and recommendation explanations, for example:
"You described lengthy preparation as unsuitable for weekdays. This recipe lists a preparation time of 20 minutes."
Compare extractive summaries with generated summaries. Evaluate factual support, correct preference attribution, and preservation of conditions. Every personalized claim should trace back to historical evidence.

10. **Annotation and Evaluation Protocol**

Begin with a 200-review pilot, then annotate approximately 800–1,200 reviews for entities, holders, aspects, sentiment, and conditions. Double-annotate a subset and report agreement.
Evaluate:

- **NER:** entity-level precision, recall, and F1.
- **Sentiment and condition detection:** macro-F1.
- **Relation extraction:** tuple-level F1.
- **Topic modeling:** coherence, diversity, and human interpretation.
- **Recommendation:** NDCG@10 and Recall@10.
- **Explanations:** evidence support and incorrect-attribution rate.

Use chronological recommendation splits and only historical reviews. Because Food.com does not directly supply the context of each future recommendation request, evaluate contextual matching separately with manually authored requests and independently assessed candidate relevance. Do not use a target review's context as if it were available before the interaction.
Expected contribution: An interpretable NLP pipeline that distinguishes **general preferences from situational preferences**, with a benchmark showing whether preserving the holder and condition improves food recommendation and explanation accuracy.

Project 30: Analyzis of Wordnet consistency in Synonymy and Antonymy relationships

This project aims to analyze some WordNet consistency or inconsistency in dealing with synonymy and antonymy relationship

Consider the list of synonyms and antonyms list available in Gutenberg available at A COMPLETE DICTIONARY OF SYNONYMS AND ANTONYMS, OR SYNONYMS AND WORDS OF OPPOSITE MEANING. by Samuel Fallows—A Project Gutenberg eBook

1. Use wordnet lexical base to retrieve the list of pairs of tokens linked by synonymy or antonymy relationships for either noun or verb categories only. In order to avoid the timely API call to for wordnet semantic similarity, it is better to retrieve from existing databases in wordnet resources, database of already saved results on semantic similarity of all pairs of words. Write a script that allows you from the database (use either path or Wu and Palmer semantic similarity) to retrieve the minimum, maximum and average semantic similarity for noun category and verb category. Repeat this process of anonym pairs as well, indicating the smallest, largest and average semantic similarity score across all pairs.
2. Repeat the calculus in 1) where one generates the histogram distribution for synonym and antonym pairs (this is generated by considering the minimum and maximum score, then consider 10 equally

bins, and counting the total number of pairs whose semantic similarity falls within a given bin limit). Plot the histograms for synonymy and antonymy relationship.

3. Discuss whether semantic similarity can distinguish between synonymy and antonymy. Consider validating your answer using statistical hypothesis testing.
4. Now we want to evaluate some fundamental properties of synonymy and antonymy relationships, especially the transitivity relationship. For instance, does it hold that if X is synonym to Y and Y is synonym to Z, then X is synonym to Z as well. Similarly, if X is antonym to Y and Y is antonym to Z, then X is synonym to Z. Write a script that allows you to test this hypothesis in both noun category and verb category.
5. We also want to test the intuitive reasoning that if X and Y are synonym then they may belong to same category, and therefore can have same root node (hypernym). Write a script that would allow you to test for each pair of synonyms, whether there is exist a configuration of synsets of the two words that have the same hypernym.
6. Repeat the above reasoning when using the same antonym relationship
7. Now we want to use the word2vec (considering only 2 dimension space, using either t-SNE projection or changing the hyperparameters of word2vec) embedding for synonyms and antonyms. Then compute the smallest and maximum distance for synonyms and antonym relationships of verbs and antonyms. Provide some graphical illustration.
8. Now we want to consider alternative database of synonymy / antonymy, A COMPLETE DICTIONARY OF SYNONYMS AND ANTONYMS, OR SYNONYMS AND WORDS OF OPPOSITE MEANING. by Samuel Fallows—A Project Gutenberg eBook. Write a script that allows you to identify examples of noun and verb pairs that are not present in Wordnet list of synonyms / antonyms. Then provide the wordnet semantic similarity of those synonyms / antonyms not present in wordnet, and discuss their compliance with the range of values identified in 1)
9. Identify relevant literature and provide critical concise review of the state-of-the-art in this area.

Project 31: WordSense Disambiguation in Medical Context

This project aims to contribute to Word Sense Disambiguation in the domains of biomedical and clinical text.

1) Initially study again Lesk implementation in NLTK. Consider the sentence “I have been prescribed two important drugs today during my visit to clinic” and assume that the target word is “drug”. Construct a simple script that outputs the correct sense of “drug” in the above sentence using standard Lesk algorithm. And, comment on the outcome the efficiency of the result.
2) Download and study MSH-WSD dataset from https://github.com/clips/yarn. The database contains 203 ambiguous terms that include regular terms, acronyms. Make a random selection of 10 acronyms and

10 regular terms. Use Lesk algorithm to perform the wordsense disambiguation of each of these 10 acronym and 10 regular terms and use the ground truth to compute the average accuracy on regular terms and average accuracy on acronyms

3) Study the various disambiguation algorithms implemented in https://github.com/alvations/pywsd. Show that you can run the program on a simple sentence containing a target word to disambiguate and output for various disambiguation algorithms (Original Lesk, adapted/extended Lesk, simple Lesk extended with hypernym and hyponyms, cosine Lesk, path-semantic similarity, Information content-semantic similarity, highest Lemma count.
4) Run the above various algorithms in 3) on MSH-WSD dataset and report the average accuracy on all acronyms and average accuracy on all regular terms.
5) We would like to extend similarity-based Lesk algorithm to include phonetic matching, which evaluates phonetically similar wording. For this purpose, use the "Fuzzy" library in python to generate the character string that identifies phonetically similar words, and then use edit distance to compute the phonetic similarity between two words. See an example of implementation at https://www.analyticsvidhya.com/blog/2017/01/ultimate-guide-to-understand-implement-natural-language-processing-codes-in-python/. Demonstrate this reasoning on the example sentence pointed out in 1) and generate a matrix that indicates the phonetic distance between each pair of words of the sentence.
6) We would like to extend the Wu and Palmer-semantic similarity extension of Lesk algorithm implemented in 3) by incorporating the phonetic similarity as well so that the overall similarity is computed as a convex combination of Wu and Palmer similarity and phonetic similarity (after being normalized) with coefficient of 0.5. Suggest and an implementation and run your new code on selected MSH-WSD dataset.
7) Suggest a GUI of your choice that allows the user to visualize the various outputs of the disambiguation task. Ideally the use can input a sentence and a target word to be disambiguated.
8) Identify relevant literature and critical analysis in this field.

Project 32: Literature mapping

The aim of this project is to use open access library search API to mine environment-related topics. For this purpose, use OpenAlex API OpenAlex: The open catalog to the global research system, which enables advanced search functionalities. We focus on a climate change. More specifically, we focus on the query "climate change mitigation"

1. Write a script that calls the Open Alex API with the query "climate change and community resilience". We focus on the search outcome at different time period:1850-1900; 1900-1920; 1921-1940; 1941-1960; 1961-1980; 1981-2000; 2001-2012; 2013-2024. For each time period, select the 30 results that have the highest citations. For each outcome create a metadata, which include the title of the paper, its abstract, the corresponding topic in the OpenAlex category, list of authors and their institution. For each time period, create a dataframe gathering all the thirty datasets.
2. Write a script that scrutinizes the categories associated to documents of the same time period, and output the percentage of each category in the thirty outcomes. Then, for a given category that

appears in more than one time period, draw a plot that shows the evolution of each category across different time periods (Trace the categories on the same plot).

3. Now we want to explore the titles of the documents. For this purpose, suggest a script that gathers all titles of the same time period, then uses DistilBERT to generate an embedding vector, then uses the t-SNE Visualizing Word Vectors with t-SNE | Kaggle, to project the embedding into a 2D space, so that all titles of the same time period are represented as a single 2D point. Finally, draw a plot showing the evolution of the titles across various time periods.
4. Repeat the process in 3) when considering all abstracts of documents falling under the same time period.
5. Now we want to comprehend the diversity of documents falling on the same period. For this purpose, use the DistilBERT embedding for each abstract, followed by t-SNE decomposition for 2D representation, and draw a plot for each time period highlighting the distribution of abstracts in the 2D plot (each abstract corresponds to a single point), at the same time, draw the distribution of categories over the same time period. Comment whether some analogies between embedding-based representation and category-based representation.
6. We want to assess the compatibility of category result outputted by OpenAlex with abstract topic-based analysis. For this purpose, given the set of all abstracts falling on the same time scale, with the corresponding embedding as performed in Task 4), create a DistilBERT embedding vector for each category title as described in OpenAlex, and then write a program that computes the cosine similarity between each topic title embedding and the abstracts embedding, so that the category that yields the highest cosine similarity score will be assigned to it. Comment whether the compatibility is feasible when taking the overall set of abstracts.
7. Repeat 6) when considering individual abstracts instead of the concatenated list of all abstracts.
8. Repeat 6) and 7) when word2vec was used instead of DistilBERT embeddings. Summarize the findings in a table highlighting the performance of various embeddings and data processing pipeline.
9. Use appropriate literature to comment on the findings. Also, identify any additional input that would allow you to further elucidate any of the preceding, and use appropriate literature of corpus linguistic literature to justify your findings and comment on the obtained results. Finally, comment on the limitations and structural weakness of the data processing pipeline

Project 32bis: Sentence Semantic Similarity

This project aims to investigate the similarity between two phrases using internet search results only.

1. We want to study a new WebJaccard similarity –see paper "WebSim: A web based semantic similarity measure" in Proc. of the 21st Annual Conference of the Japanese Society of Artificial Intelligence, 2007. More specifically, for two words P and Q, the similarity Sim(P,Q) is calculated as follows

   $$Sim(P, Q) = N11 / (N11 + N10 + N01)$$

   where N11 corresponds to the page count for the query "P AND Q), or equivalently, the number of results when the "P AND Q" is inputted to the search engine (i.e., Google search, yahoo search). N10 corresponds to the page count of the query "P AND NOT Q". N01 corresponds to the total page count of the query "B AND NOT A". Suggest a script that implements the above Web-based semantic similarity using Google Search API or any other search API.

2. Now we would like to see how the above webSimilarity behaves when dealing with popular corpuses of manually annotated pair of keywords. For this purpose, we consider MC (Miller and Charles, 91), RG (Rubenstein and Goodenougth, 1965), WordSim353 (Finkelstein et al., 2001), that are available, for instance, from GitHub - alexanderpanchenko/sim-eval: A tool for evaluation of semantic similarity measures. · GitHub. Note that the score is not necessarily normalized among these datasets. Suggest a script that uses the web search to record the number of search results when one single term of the pair is input and when both terms were inputted, and then generate the webJacccard similarity value for each pair. Now for each dataset, consider the vector containing the annotator result for the given pair of word, and another vector containing the WebJaccard similarity for the same pair, and then compute the Pearson correlation coefficient between the two vectors. Perform this process for MC, RG, WS (WordSim353) datasets.

3. Use wordnet library to calculate the Wu & Palmer, path length and Leacock Chodorow semantic similarity of the pairs in the above datasets, and calculate Pearson correlation between each given similarity vector and the human annotator data. Summarize the result in a table and comment on the compatibility WebJaccard, and WordNet semantic similarities with human annotation.

4. Repeat Task 3 when the similarity between the pair of results is computed as the cosine similarity between the embedding vector of each word in the pair. You may compare word2vec, FastText, Glove, DistilBERT embedding for this purpose.

5. Now we want to use Snippet instead of page count. For this purpose, use Google Search API (should register to get a key), and collect snippet of the search outcome, see an example in https://www.thetopsites.net/article/52274742.shtml. Provide a small pseudo code showing your implementation steps and an example of an answer.

6. Consider a selection from the pairs suggested in 2). Retrieve the first five snippets for query P and the first five snippets for query Q. Then one assumes that Sim(P,Q) is calculated as the ratio of the common words between the five snippets of P and the five snippets of Q over the total number of distinct words of all the ten snippets, after appropriate tokenization, stemming and noise removal

in the original text. Compare the results with that of wordnet similarities and comment on the appropriate. Assume such similarity is called Sim_Snippet1. Perform this operation for MC and RG dataset and compute the Pearson correlation with the human annotation as well.

7. Consider three queries that bear contrasting effect, e.g., “love” and “hate”; “pollution”, “eco-friendly”; “sustainable”, “unsustainable”. For each query (of the six queries), retrieve the first 10 snippets. For each contrasting query, report i) the number of snippets which are shared between the two queries; ii) the amount of overlapping between the two set of snippets. For the purpose of ii), you may concatenate all snippets of the first query into a single document D1 and that of the second query into another document D2, then use fuzzy-string matching https://www.datacamp.com/community/tutorials/fuzzy-string-python to generate the percentage of the overlapping. The latter uses edit distance and takes into account all possible alignments of D1 and D2. Summarize the result of the three double queries in a table. Comment on whether Google search tends to discriminate or re-assemble contrasting queries. The newly constructed similarity is called Sim_Snippet2.
8. Apply Sim_Snippet2 to the same examples of MC and RG datasets and compute the Pearson correlation coefficient.
9. Suggest your own reasoning pipeline to implement an LLM-based strategy of your choice for searching the similarity between each pairs of words. Compare prompting strategies where few examples from Wu and Palmer are given as examples to LLM (zero short learning versus few shot learning).
10. Design a simple GUI of your choice that shows the implementation of WebJaccard, semantic similarities, Snippet based similarity and LLM-based similarity.
11. Use appropriate literature to comment on the findings. Also, identify any additional input that would allow you to further elucidate any of the preceding, and use appropriate literature of corpus linguistic literature to justify your findings and comment on the obtained results. Finally, comment on the limitations and structural weakness of the data processing pipeline.

Project 33: Semantic Similarity 2

We aim in this project to study a new semantic similarity for two short text document using state-of-the-art transformer models and study effects of various denoising. Consider two sentences S1 and S2 which are tokenized for instance, say $S_1 = (w_1, w_2,..,w_n)$ and $S2= (p_1, p_2, \ldots, p_m)$. We also consider standard wordnet semantic similarity measures.

1. Use publicly available dataset (pair of sentences) STSS-131 dataset, available in "A new benchmark dataset with production methodology for short text semantic similarity algorithms" by O'Shea, Bandar and Crockett (ACM Trans. on Speech and Language Processing, 10, 2013). Use Pearson correlation coefficient to test the correlation of your result with the provided human judgment. First consider the Wu and Palmer semantic similarity where the sim(S1, S1) is calculated as

$$Sim(S_1, S_2) = \frac{1}{2}\left(\max_{S_1^{ni} \in S_1, S_2^{nj} \in S_2} Sim_{WP}(S_1^{ni}, S_2^{nj}) + \max_{S_1^{vi} \in S_1, S_2^{vj} \in S_2} Sim_{WP}(S_1^{vi}, S_2^{vj})\right) \quad (1)$$

   Where S_1^{ni} (S_1^{vi}) stand for noun tokens (verb tokens) of sentence S1. Similar remark for S_2.

   Use (1) to calculate similarity of all sentence pairs in STSS-131 and calculate the Pearson correlation score with human judgement (available in dataset).

   Repeat this process using other publicly available datasets: SemEval STS datasets (2012–2017), the STS Benchmark (STSB), SICK-R, and MRPC, see Sentence representations for semantic textual similarity: A systematic review - ScienceDirect. Summarize the result in a table.

2. Instead of (1), we want to use the word2vec embedding, where Sim(S1, S2) is the cosine similarity of embedding vectors of sentence S1 and S2, where the embedding vector of a sentence is the average of embedding vectors of all its tokens, without any processing. Calculate the corresponding Pearson correlation. Repeat this process other datasets and summarize the result in a table.

3. Repeat 2) when Fastext embedding is using, when Sentence-BERT, RoBERTa, ALBERT embedding is used. Note that for BERT embedding all tokens are entered at once for each sentence to generate the embedding vector (no need for average over all tokens). Summarize the results in a table.

4. We want to test the robustness of the similarity. For this purpose, repeat 1-3) when preprocessing is performed prior of calculus of semantic similarity. Discuss the various options for preprocessing (e.g., selected list of stopword, without/with lemmatization), Summarize the result in a table.

5. Consider a subset of each dataset where you select 5 random pairs of sentences from each dataset. Then choose a verb, adjective / adverb at random from each sentence and replace it by its synoym (check manually the sentence to ensure that it bears the same meaning as the former sentence), then repeat 1-3) for these subset only and report the results in a table.

6. We want to disturb the meaning of the subset of pairs in 5) by injecting a negation in one of the sentences of the pair (manually ensure a shift in the meaning), then calculate the similarity of the

two sentences using each of the similarity measures tested previously. We want to report the difference between the similarity of sentence pairs before introducing the negation and after the negation. Intuitively, the similarity measure that yields larger gap in terms of similarity measures is deemed more important. Analyze the performance of the various similarity measures from this perspective.

7. Consider the other datasets in SemEval2024 Task1, see GitHub - semantic-textual-relatedness/Semantic_Relatedness_SemEval2024: SemEval 2024 Task 1 : Textual Semantic Relatedness · GitHub: Test two methods that showed good results from your previous manipulation, and summarize the findings in a table.
8. There are several implementations available for Semeval2024, investigates some of the top performing models and suggest an approach that capitalize on your previous findings to devise some hybrid methods (e.g., ensemble learners or convex combinations or any other forms using your own self initiative) and compare the performance on both SemEval dataset and those used in 1—3).
9. Suggest an interface of your choice that would allow the user to input a textual query in the form of a pair of sentences and output the similarity score according to the various methods described above.
10. Use appropriate literature to comment on the findings. Also, identify any additional input that would allow you to further elucidate any of the preceding, and use appropriate literature of corpus linguistic literature to justify your findings and comment on the obtained results. Finally, comment on the limitations and structural weakness of the data processing pipeline.

Project 34 Sentiment Analysis on Tweets with Transformer Models

The goal of this project is to perform sentiment analysis on a dataset of tweets with additional demographic and global context features, including user age, country, and population statistics. This project will explore the use of state-of-the-art transformer-based models to analyze sentiment while incorporating metadata such as tweet time, user demographics, and country data to provide a more holistic sentiment analysis. Sentiment Analysis Dataset Sentiment Analysis Dataset contains the following fields: textID, text, selected_text, sentiment, Time of Tweet, Age of User, Country, Population 2020, Land Area (Km²), and Density (P/Km²).

1. Data Exploration and Preprocessing: Explore the distribution of tweets by sentiment, Age of User, Country, and Time of Tweet. Visualize the data using appropriate charts such as bar charts, histograms, and heatmaps.

2. Exploratory Analysis of Demographics: Analyze the correlation between demographic information (e.g., Age of User, Country) and the sentiment of the tweets. Identify if certain age groups or countries tend to express more positive, negative, or neutral sentiments. Also. analyze how density affects sentiment patterns.

3. **Time of Tweet and Sentiment Trends**: Investigate how the time of tweet (morning, noon, night) correlates with sentiment. Do people tend to tweet more positively or negatively at certain times of the day? Visualize sentiment trends over time to identify any patterns (e.g., higher negativity during late hours or weekends).

4. Topic Modeling with LDA on Tweets: The main aim of this task is to use LDA (Latent Dirichlet Allocation) to discover hidden topics within the tweet dataset and analyze how these topics correlate with different sentiment classes (positive, neutral, negative). Apply LDA to the *text* or *selected_text* field to identify the most common topics discussed in the tweets. After extracting the topics, categorize tweets based on the most likely topic and analyze the distribution of sentiment across different topics. Visualize the topics using word clouds or bar charts to interpret the most frequent words associated with each topic.

5. Coherence-Based Topic Model Tuning: The purpose of the task is to optimize LDA's performance by tuning the number of topics based on topic coherence. Experiment with different numbers of topics for LDA (e.g., 5, 10, 15, 20) and use a coherence score (e.g., CV coherence) to evaluate the quality of the topics. Fine-tune the number of topics based on coherence score to identify the most meaningful number of topics for the dataset.

6. Word Embeddings for Topic Similarity: The aim of this task is to use Word2Vec or BERT-based embeddings to analyze the similarity between different topics extracted via LDA (Task 4). Generate word embeddings for the most frequent words in each LDA topic using Word2Vec or BERT. Calculate the cosine similarity between different topics based on their word embeddings to see which topics are semantically similar. Visualize the similarity between topics using heatmaps or dendrograms.

7. Topic Sentiment Evolution with Transformer Embeddings: In this task we want to combine transformer-generated embeddings (e.g., BERT embeddings) with LDA to improve topic extraction and sentiment analysis. Generate sentence embeddings for each tweet using a transformer model like BERT. Use these embeddings as input for an LDA variant that works with dense vectors (e.g., BERTopic). Analyze the resulting topics and their sentiment evolution, paying close attention to how transformer embeddings impact the quality and coherence of topics.

8. Sentiment Analysis Using Transformers: Leverage pre-trained transformer models such as BERT, RoBERTa, and DistilBERT for sentiment classification (positive, neutral, or negative). Fine-tune these models on the train dataset using the text and selected_text (which represents the portion of the tweet most relevant to the sentiment) columns separately. Then, conduct fine-tuning by training on the labeled sentiment data on train data and test the result on Test data. Additionally, experiment with different variants of BERT (e.g., BERT-large vs. BERT-base) to improve accuracy. Compare the performance of transformer models (BERT, RoBERTa) against other NLP techniques, such as TF-IDF + Logistic Regression, to demonstrate the advantage of transformers in this sentiment classification task.

9. Multi-modal Analysis with Metadata: Incorporate user demographic features (Age of User, Country, Population 2020, Density (P/Km²)) along with tweet text in the model. Train a multi-input model where one branch processes the textual content (using transformers) and another branch processes demographic and geographical features of the users. Combine the results to make a final

sentiment prediction. Evaluate the importance of each feature using SHAP values or feature importance analysis to understand the contribution of text versus demographic data to the overall sentiment prediction.

10. Sentiment Prediction Using Pre-trained LLMs: Apply LLMs like GPT to classify tweet sentiment by fine-tuning them on this dataset. These models, with their ability to capture nuanced contextual relationships, could outperform smaller transformer models on more complex sentiment tasks. For models like GPT, experiment with prompt engineering to extract sentiment in a zero-shot or few-shot learning setting, where the model makes predictions based on minimal examples. Finally. compare the sentiment classification performance of fine-tuned BERT,and LLM using accuracy measures.

11. Error Analysis and Improvements: Perform error analysis by inspecting tweets where the sentiment prediction was incorrect. Look for patterns in errors, such as sarcasm, complex language, or ambiguity. Optionally experiment with LLMs such as GPT for more advanced contextual understanding and sentiment classification. Compare their performance with transformer-based models like BERT or RoBERTa.

12- Advanced Transformer and LLM techniques and Suggestions: Feel free to analysis and explore additional Transformer and LLM techniques or state-of-the-art models to improve the sentiment analysis task. You can experiment with other state of the art models, comparing their performance to the previously used methods.

13- Identify appropriate literature to discuss the findings and comment on the strength and weakness of the data processing pipeline.

Project 35: Semantic Product Matching for Finnish B2B Quotation Requests (Reserved to Samu Juhani Lepistö and Dominik Maier)

Goal:

Create a retrieval system that accepts a Finnish customer request line as a free text description and returns a ranked list of matching products with similarity scores from the catalog of a technical B2B wholesaler. A comparison of real public requests for quotation reveals the effectiveness of various approaches, including keyword embedding, hybrid methods, reranking, and retrieval augmentation.

Dataset:

The public online store catalogue contains approximately 74,000 products in CSV format, with each product having a name, code, product hierarchy (code and label), HTML-tagged long description, and clean short text. All text is in Finnish. Product titles are concise and comprise acronyms, codes, and technical specifications, such as "M8x40" or "A4".

The test data consists of previous public requests for quotation (RFQ) issued by public organizations such as cities, municipalities, and public institutions. The data is in PDF or Excel format. For each RFQ, the wholesaler's response is available as an Excel file linking each request line identifier to the offered product code and name. The offered product is the gold answer. The requests and responses are devoid of any pricing or other sensitive information, which means that all data can be made public along with the code.

Suggested scope:

There are 150 to 200 request lines from 10 to 15 public RFQs. These are split into a development set (one third) and an evaluation set (two thirds). The sets are complemented with 30 to 50 manually authored queries for underrepresented product categories. All processing runs locally on a laptop with 16 GB of memory.

Research question:

Which existing or project-developed retrieval method is most effective at identifying the correct product for a Finnish customer request? Does the answer differ between standardized products (DIN, ISO) and harder categories, such as hand tools, clothing, and cutting tools?

Keyword searches struggle with Finnish inflection, compound words, and different product descriptions, while dense retrieval may lose exact codes and dimensions. Therefore, testing where each approach fails is as important as measuring overall accuracy.

Specifications:

1. **Dataset exploration and preparation:** Analyze the distribution of products across the product hierarchy, as well as the lengths of titles and descriptions, vocabulary size, and the frequency of codes, units, and abbreviations. Verify whether the product vocabulary follows Zipf's law. HTML tags should be removed, dimensions and units should be normalized (for example, "M8X40", "m8 x 40" and "M8 x 40 mm" should be converted into one form) and a single searchable text field should be constructed per product.
2. **Test set construction:** Write a script that extracts the request lines and their identifiers from RFQ files using Pandas for Excel and Pdfplumber for PDF. Join the extracted lines with the response files using the line identifiers. Store the results in database Q, which has the following attributes: RFQ identifier, line

identifier, issuing organization type, request text, gold product code, gold product name, and product category. The category is taken from the product hierarchy of the gold product. Verify that every Gold code exists in the current catalog. Report lines that could not be joined and lines for which no product was offered. Also report offered products that are no longer available in the online store. Gold answers must be kept hidden from the retrieval systems.

3. **Keyword retrieval baselines:** Implement BM25 on raw tokens and after Finnish Snowball stemming. Also, implement TF-IDF on character n-grams of three to five characters. Then, compare how each variant handles inflected forms, compound words, and product codes.
4. **Dense retrieval:** Use a multilingual sentence embedding model (Multilingual-E5-Base, or BGE-M3 as an alternative) to encode all products directly in Finnish, and store the vectors with product attributes as metadata in a vector database (Qdrant in local mode). Explain why the Finnish text is embedded directly instead of first translating the catalog and requests into English.
5. **Hybrid retrieval:** Use Reciprocal Rank Fusion to combine the best keyword ranking and dense ranking. Only adjust the fusion parameter on the development set.
6. **Reranking:** Use a cross-encoder (bge-reranker-v2-m3) to rerank the 20 best hybrid candidates and measure the increase in accuracy compared to the additional latency.
7. **Retrieval augmented selection:** Implement an RAG pipeline in which the twenty best hybrid candidates, along with their names, brief descriptions, and hierarchy labels, are provided as context to a local LLM (e.g., a 7B to 8B model via Ollama). The model must return the selected product code in a fixed JSON schema along with a short justification referring to attributes in the request. The model should also return the value "none" when no candidate fits. Verify that each returned code exists among the candidates. Then, compare the results of the RAG pipeline with those of the cross-encoder reranking on the same candidate lists in terms of Hit@1, latency, and correct abstention.
8. **Evaluation and comparison matrix:** Write a script that processes each request line in Q through every method and outputs a matrix. The rows of the matrix should be the request lines and the columns should be the methods. Each cell should contain the top-ranked product code, name, and score next to the correct answer. Report Hit@1, Recall@5, Recall@10, MRR, query latency, and indexing time, overall and per product category. Use McNemar's test to test whether differences between methods are significant for Hit@1 and bootstrap confidence intervals for MRR.
9. **Error analysis:** Since the response lists a single offered product per request line, manually inspect the failed top 1 predictions and categorize them as abbreviation errors, dimension or number mismatches, wrong product category, or acceptable substitutes. Then, discuss what the categories reveal about the strengths of keyword and semantic matching.
10. **Graphical interface:** Design a Gradio interface in which users can type in a product description and see the top five results from each method side by side. The interface should also allow users to upload an Excel RFQ to produce a comparison matrix that can be downloaded as a file.
11. **Literature review:** Give a brief and analytical overview of the key studies on product search, hybrid sparse and dense retrieval, retrieval augmented generation, and retrieval for morphologically rich languages. Then, use this overview to interpret the results.
12. **Deliverables:**

    - Cleaned product database.

- Test set Q.
- Retrieval indexes.
- RAG prompt templates.
- LLM output logs.
- Evaluation notebook.
- Comparison matrix.
- Gradio application.
- Comparison report.

Optional extensions:

- The rule-based extraction of attributes (product type, dimensions, material, and standard) is used as a metadata filter in the RAG pipeline. There is a comparison of the local LLM with a larger, API-based model on the evaluation set. There is also a PDF upload in the interface.
- Testing a local weight decision model for RAG implementation, as like a Laya.

Important experimental principle: Better performance of the embedding search and RAG pipeline is a hypothesis to test. A well-supported finding is that a simple keyword baseline matches or beats them on short technical titles, which is also a successful project outcome.

Project 36: JudgeRec: Selective LLM-as-a-Judge for Efficient and Reliable Recommendation Evaluation [Reserved to Syed Mahim]

Project Overview

Main research question: Can personalized recommendations be evaluated using a low-cost LLM judge for routine cases while selectively escalating uncertain or difficult cases to a stronger reasoning judge, without materially reducing agreement with human judgments?

Core mechanism: Recommendation → Cheap Judge → Confidence → Accept if confident / Escalate if uncertain.

The main scientific contribution is to determine when recommendation evaluation actually requires expensive reasoning and whether uncertainty can be used to allocate evaluation compute selectively.

Overall Pipeline

1. Dataset construction
2. Recommendation generation
3. Evaluation-instance construction
4. Cheap first-stage judge
5. Confidence estimation and calibration
6. Selective escalation
7. Strong reasoning judge
8. Human validation and cost-quality analysis

Step 1 — Construct Recommendation Evaluation Instances

Define each evaluation instance as E = (U, H, C, I), where U is the user profile, H is interaction history, C is the current context, and I is a candidate recommendation.

Example instance: User frequently chooses vegetarian recipes and prefers meals under 30 minutes. Recent history includes vegetable pasta, lentil curry, and mushroom risotto, while tofu salad was disliked. Context: weekday evening. Candidate: spinach ricotta pasta, 28 minutes, vegetarian. Task: Is this a good personalized recommendation?

Recommended scenario types:

- Easy positive: strong preference match
- Easy negative: clear mismatch
- Ambiguous: partial match

- Sparse profile: little interaction information
- Preference drift: recent behavior differs from older behavior
- Long-term vs. short-term conflict: candidate is liked but conflicts with another stated objective

Specified output: a benchmark of 3,000–10,000 evaluation instances with user history, candidate item, context, and scenario/difficulty label.

Step 2 — Generate Recommendations from Multiple Systems

Use multiple recommendation systems so that the judge is exposed to outputs of different quality and modeling assumptions.

- Popularity baseline
- Collaborative filtering or matrix factorization
- Sequential recommender such as SASRec
- Semantic/content-based recommender
- Prompt-based LLM recommender
- Personalized LLM recommender using verbalized user history
- Random control for clearly poor recommendations

Each recommender produces a Top-K list R(u) = [i1, …, iK].

Specified output: a candidate pool with columns such as user_id, recommender, rank, and item_id.

Step 3 — Define the Recommendation Judgment Task

Use structured outputs instead of unrestricted explanations for the primary task. Pairwise recommendation evaluation is recommended as the main formulation because it avoids instability associated with absolute scoring.

Judge task: Given a user profile, recent history, current context, and two candidate items, determine which item is a better recommendation.

Allowed outputs: A / B / Tie, together with P(A), P(B), and P(Tie), constrained to sum to 1.

Specified output: a standardized Recommendation Judge Contract: J(E) = (Verdict, P(A), P(B), P(Tie)).

Step 4 — Build the First-Stage Cheap Judge

Use an inexpensive model as J_fast. Candidate options include a small open-weight LLM, a low-cost API model, a specialized reward model, or a fine-tuned small LLM.

For each instance, compute the class probabilities and define confidence as $q = \max_k P(k)$.

Example: $P(A)=0.91$, $P(B)=0.06$, $P(Tie)=0.03$, so $q=0.91$.

Specified output: for every instance, store the fast-judge verdict and confidence.

Step 5 — Calibrate Judge Confidence

Confidence should be treated as an empirical quantity rather than assumed to be reliable. A predicted confidence of 0.90 does not automatically mean 90% correctness.

Use a separate human-labeled calibration set, for example 200–500 examples. Apply calibration methods such as temperature scaling or isotonic regression.

Evaluation metrics:

- Expected Calibration Error (ECE)
- Brier score
- Negative log-likelihood (NLL)
- Error-detection AUROC: whether low confidence predicts incorrect judgments

Output: calibration curves, raw-vs-calibrated confidence metrics, calibrated probability model for selective routing.

Step 6 — Selective Escalation

Use a confidence gate: if $q \geq \tau$, accept the fast judge; otherwise escalate the case to a stronger reasoning judge.

Test thresholds $\tau \in \{0.50, 0.60, 0.70, 0.80, 0.90, 0.95, 0.99\}$.

Compare different second-stage models, e.g., a larger generic LLM versus a reasoning-optimized LLM.

Specified output: for each example, record confidence, routing action, final judge, and final verdict. Also compute Escalation Rate = #Escalated / N.

9. Step 7 — Human Ground Truth

Human evaluation is required for a convincing paper. A target of roughly 500–1,000 pairwise judgments is recommended if feasible, ideally with two or three raters per case.

Two evaluation tracks can be used:

- Track A — User-grounded cases: use real acceptance, rejection, rating, choice, or other direct user feedback when available.
- Track B — Annotator-grounded cases: provide the user profile and ask annotators which recommendation better matches the user.

Use blind model identities and randomized candidate order. Report inter-annotator agreement using Krippendorff's alpha or Fleiss' kappa.

Do not automatically discard human disagreement. It may indicate intrinsic recommendation ambiguity and can itself be compared with judge uncertainty.

Specified output: a human-validated benchmark subset with consensus labels and human-human agreement statistics.

Step 8 — Main Evaluation

Compare the following strategies:

- Baseline 1 — Cheap judge only
- Baseline 2 — Strong judge only
- Baseline 3 — Random escalation at the same escalation rate as JudgeRec
- Baseline 4 — Heuristic difficulty routing using simple signals such as short history or small candidate-score differences
- Method — Confidence-based selective judge: J_fast → J_strong

Core Evaluation Metrics

Judgment Quality

- Accuracy
- Macro-F1
- Pairwise agreement with humans
- Cohen's kappa or similar chance-corrected agreement

Ranking-Level Agreement

- Kendall's tau between judge and human rankings
- Spearman correlation
- NDCG@K where human preference defines relevance

11.3 Selective Prediction Quality

Plot risk versus coverage. Coverage is the fraction of cases accepted by the cheap judge; risk is one minus the accuracy among accepted cases.

The desired pattern is high coverage with low risk. This directly tests whether the model is trustworthy when it claims confidence.

Efficiency

- Total API cost
- Input and output tokens
- Median latency (p50)
- Tail latency (p95)
- Escalation rate
- Cost saving relative to strong-judge-only evaluation

Cost saving can be expressed as: $1 - \text{Cost}_{\text{selective}} / \text{Cost}_{\text{strong}}$.

Calibration and Error Detection

- ECE
- Brier score
- NLL
- Error-detection AUROC
- Recall of cheap-judge errors captured by escalation

Key Result Figure — Cost–Quality Frontier

The main summary figure should plot Relative Cost on the x-axis and Human Agreement on the y-axis. Include cheap judge, random escalation, heuristic escalation, multiple JudgeRec thresholds, and strong judge. The key question is whether selective judging lies on a better cost-quality frontier than these alternatives.

Step 9 — Recommendation-Specific Difficulty Analysis

Analyze whether selective reasoning provides different value across recommendation conditions.

- Strong preference match
- Strong preference mismatch
- Sparse history
- Mixed historical preferences
- Preference drift

- Near-equivalent candidate items
- Popularity conflict
- Long-term versus immediate preference conflict

Specified output: for each scenario, report fast-judge accuracy, average confidence, escalation rate, strong-judge accuracy, and final selective-judge accuracy.

Step 10 — Confidence Failure Analysis

Identify recommendation-specific cases where the cheap judge is confidently wrong. This should be treated as a major analysis rather than an appendix-only result.

- Popularity bias
- Semantic-similarity bias
- Explanation/persuasion bias
- Profile-length bias
- Recency bias
- Candidate-order bias

For pairwise evaluation, test both (A,B) and (B,A) orders and measure position-flip rate.

Specified output: a taxonomy of high-confidence judge failures in recommender-system evaluation.

Step 11 — Threshold Selection

Do not select τ using the test set. Use separate calibration, validation, and test splits.

A useful operating-point objective is: choose the cheapest threshold such that selective-judge accuracy is no more than ε below strong-judge accuracy. For example, $\varepsilon = 0.01$.

This turns threshold selection into a constrained optimization problem instead of an arbitrary hyperparameter choice.

Specified output: a fixed thresholding policy determined before final test evaluation

Project 37: Sentiment on Technically Dept dataset

The objective of the project is to evaluate the Research question RQ: Does sentiment analysis have an impact on Technical Debt Detection? Does negative sentiment from source code comment contribute to technical debt?

Dataset: https://github.com/maldonado/tse.satd.data

Manually labeled 62,275 source code comments, of which

Tech Debt (SATD) comments: 4071 – Category X

Design Debt : S1 subclass

Implementation Debt : S2

Defect Debt : S3

Test Debt : S4

Documentation Debt : S5

Non Tech Debt (Non SATD) comments: 58204 – Category Y

1. We want to comprehend each categories in terms of wording they do contain. For this purpose, group comments pertaining to the same category in the same document, perform appropriate filtering and lemmatization, then plot WordCloud representation of the associated file. Draw a histogram of the twenty frequent words in each category.

2. We want to find out if one can characterize each category in terms of unique words it contain. For this purpose, after appropriate preprocessing and lemmatization, construct a vocabulary S of the common words that appear in all categories, and vocabulary S1, S2, S3, S4, S5, S6 of wording that were uniquely present in a given category not in any other categories. Discuss the cardinality of S, S1, S2, S3, S4, S5 and S6. Draw histogram showing the frequency of the unique wording in each category. Store S, S1, S2, S3, S4, S5 and S6 in a separate database, say D1.

3. Use LDA with topics equal 6 and 20 words per topic. We would like to find out whether output of the LDA matches the categorization provided by word frequency or unique wording. For this purpose, for each topic, try to match the words of the topics to the twenty most frequent words and use a simple string matching (i.e., Fuzzy-string matching) to report the percentage of string matching between two sets. We shall proceed by elimination; namely, for each category characterized by its twenty most frequent words, we compute the fuzzy-string matching with every topic and the assignment is made to the topic that achieves the highest score. Discuss on the usefulness of this approach.

4. Repeat the reasoning carried out in 3) when one assumes that each category is represented by its set of unique wording S1, S2, …, S6.

5. We want to discriminate the categories using information about Empath Client. For this purpose, perform Empath Client on the document associated to each category and generate the new

categorization of each class. Suggest how you can perform to identify classes that present ambiguity and those that offer good discrimination power.

6. Now, we want to analyze the content of the categories in terms of sentiment. Use the SentiStrength from http://sentistrength.wlv.ac.uk/ to determine the positive, negative and overall (sum of positive and negative) sentiment score for each source code comment. Provide a database D1, which contains these information for each source code comment belonging to categories. Determine the average S5 and S6 (or Y). Discuss whether the sentiment score can be used as a relevant attribute to discriminate among the various categories.
7. For each category, discriminate among the comments pertaining to positive sentiment and those negative sentiment and concatenate all positive comments together and negative comments together as well. For each category, draw histogram of most frequent wording in each positive and negative sentiment. Comment whether you can use frequency distribution to discriminate among the categories.
8. We would like to test the various transitions generated by the sentiment analysis. More specifically, we count the number of cases where an overall positive sentiment is followed by a positive sentiment for the next followed comment. Summarize in a table, for each category, the number (or proportion) of cases of (Pos→Pos, Pos→Neg, Neg → Neg, Neg → Neu, Neu→Neu, Pos→Neu, Neu→Neg, Neu→Pos). For neutral (Neu), you should consider some threshold so that the overall sentiment score is between [-□ +□] where □ is chosen close to zero (e.g., 0.1, 0.2). Similarly, comment whether the transition distribution is relevant to discriminate among the various categories.
9. Use XGBoost machine learning algorithm for Technical Debt detection. Compute Precision, Recall and F1 score. Then apply sentiment based Technical Debt detection along with XGBoost machine learning algorithm. Compute Precision, Recall and F1 score. Evaluate if the negative sentiment could increase the failed detection of Technical Debt by machine learning algorithm. Compare the F1 scores of machine learning algorithm and machine learning algorithm augmented by sentiment analysis. Tabulate the results for deliverable.
10. Use RoBERTa deep learning algorithm for Technical Debt detection. Compute Precision, Recall and F1 score. Then apply sentiment based Technical Debt detection along with RoBERTa. Compute Precision, Recall and F1 score. Evaluate if the augmented sentiment based Technical Debt detection has improved the F1 score. Tabulate the results for deliverable.
11. Suggest your own initiative using recent advances in codeLLM to suggest your own approach to enhance the categorization and provide the result in table with comparison with previous results.
12. Design a simple GUI of your choice that show the execution of each of the above tasks in a way to ease the task of the assessor or external end-user
13. Identify relevant literature to comment on the finding and provide a summary of critical analysis of state-of-the-art in this field.

Resources for Finnish text analysis

Turku Dependency Treebank

Turku Dependency Treebank (TDT) was created by Turku BioNLP Group. It was rst released as a stand-alone version, but it has been integrated into Universal Dependencies project in 2015 - http://universaldependencies.org/-. Turku Dependency Treebank is shared with CC BY-SA 4.0 license.

Finnish stopwords

There are several collections of Finnish stopwords available on the internet. One extensive list is provided by University of Neuchâtel on their website (http://members.unine.ch/jacques.savoy/clef/) . Stopword lists are usually used for removing regularly used words.

FinnPos

FinnPos (https://github.com/mpsilfve/FinnPos) is an open-source morphological tagging and lemmatization toolkit for Finnish. FinnPos is released under Apache Software License v2.0.

FinnWordNet

FinnWordNet (http://www.ling.helsinki.fi/en/lt/research/finnwordnet/) is a lexical database for Finnish. It is a semantic network, linked through relations such as synonymy and antonymy. The first version was a direct translation of Princeton's English WordNet. Current FinnWord-Net version 2.0 was released in 2012 under CC-BY 3.0 license and is also a subject to original Princeton WordNet license.

FinTWOL

FinTWOL is a morphological parser for Finnish (www.lingsoft.fi). It is a product developed by Lingsoft.

OMorFi

OMorFi is an open-source morphological analyzer for Finnish language. It is freely available for download (https://github.com/flammie/omorfi) and is released under GNU GPL v3 license.

Word list of modern Finnish

The institute for the languages of Finland, Kotus, has created a word list of modern Finnish words in 2007, see (http://kaino.kotus.fi/sanat/nykysuomi/). The list contains 94110 Finnish words in their basic forms along with the in inflection types encoded in UTF-8 and stored in XML-form. The list is released under GNU LGPL (Lesser General Public License), EUPL v.1.1 and CC BY 3.0 ND. Kotus has also published a wide variety of other Finnish resources (https://www.kotus.fi/aineistot), that may be helpful when doing text mining in other research areas.

Translation API

For short texts and low usage, Google translation API can be used. However for large dataset, since Google API becomes not costly, an alternative is to use free translation API like

http://py-googletrans.readthedocs.io/en/latest/